



计 算 机 科 学 丛 书

# 嵌入式系统导论

## CPS方法

(美) Edward Ashford Lee Sanjit Arunkumar Seshia 著 李实英 贺蓉 李仁发 译  
加州大学伯克利分校 加州大学伯克利分校

**Introduction to Embedded Systems**  
A Cyber-Physical Systems Approach

### Introduction to Embedded Systems

A Cyber-Physical Systems Approach

Edward Ashford Lee  
Sanjit Arunkumar Seshia

UC Berkeley

First Edition

<http://LeeSeshia.org>



机械工业出版社  
China Machine Press

计算机科学丛书

# 嵌入式系统导论

CPS 方法

Introduction to Embedded Systems  
A Cyber-Physical Systems Approach

(美) Edward Ashford Lee 著  
加州大学伯克利分校

Sanjit Arunkumar Seshia  
加州大学伯克利分校

李实英 贺 蓉 李仁发 译

HZ BOOKS  
华章图书



机械工业出版社  
China Machine Press

本书是业界第一本关于 CPS 的专著，重点论述系统模型与系统实现的关系，以及软件和硬件与物理环境的相互作用。

从 CPS 的视角，围绕系统的建模、设计和分析这三个方面，本书分成四大部分。第一部分(第 2~6 章)分别讲述动态建模、离散建模和混合建模，以及状态机的并发组合与并发计算模型。第二部分(第 7~11 章)强调嵌入式系统中处理器、存储器架构、输入和输出、多任务处理和实时调度的算法与设计，以及这些设计在 CPS 中的主要作用。第三部分(第 12~15 章)重点介绍一些系统特性的精确规格、规格之间的比较方法、规格与产品设计的分析方法以及嵌入式软件特性的定量分析方法。第四部分包括两个附录，提供了一些数学和计算机科学的背景知识，有助于读者加深对文中所介绍知识的理解。本书通过大量实例深入浅出地介绍了设计和实现 CPS 的整体过程及各阶段的细节。

本书适合作为高等院校相关专业“嵌入式系统”课程的教材或教学参考书。

Authorized translation from the English language edition entitled Introduction to Embedded Systems—A Cyber-Physical Systems Approach, First Edition(ISBN 978-0-557-70857-4) by Edward Ashford Lee, Sanjit Arunkumar Seshia, Copyright © 2011 by Edward Ashford Lee, Sanjit Arunkumar Seshia.

Chinese simplified language edition published by China Machine Press.

Copyright © 2012 by China Machine Press.

此版本仅限在中华人民共和国境内(不包括中国香港、澳门特别行政区及中国台湾)销售。未经授权的本书出口将被视为违反版权法的行为。未经出版者预先书面许可，不得以任何方式复制或发行本书的任何部分。

封底无防伪标均为盗版

版权所有，侵权必究

本书法律顾问 北京市展达律师事务所

本书版权登记号：图字：01-2011-3604

图书在版编目(CIP)数据

嵌入式系统导论：CPS 方法/(美)李(Lee, E. A.)，(美)塞希阿(Seshia, S. A.)著；李实英等译．—北京：机械工业出版社，2011.12

(计算机科学丛书)

书名原文：Introduction to Embedded Systems—A Cyber-Physical Systems Approach

ISBN 978-7-111-36021-6

I. 嵌… II. ①李… ②塞… ③李… III. 微型计算机—系统设计 IV. TP360.21

中国版本图书馆 CIP 数据核字(2011)第 200947 号

机械工业出版社(北京市西城区百万庄大街 22 号 邮政编码 100037)

责任编辑：夏平

印刷

2012 年 1 月第 1 版第 1 次印刷

185mm×260mm·17 印张

标准书号：ISBN 978-7-111-36021-6

定价：55.00 元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换

客服热线：(010)88378991；88361066

购书热线：(010)68326294；88379649；68995259

投稿热线：(010)88379604

读者信箱：hzsj@hzbook.com

文艺复兴以降，源远流长的科学精神和逐步形成的学术规范，使西方国家在自然科学的各个领域取得了垄断性的优势；也正是这样的传统，使美国在信息技术发展的六十多年间名家辈出、独领风骚。在商业化的进程中，美国的产业界与教育界越来越紧密地结合，计算机学科中的许多泰山北斗同时身处科研和教学的最前线，由此而产生的经典科学著作，不仅擘划了研究的范畴，还揭示了学术的源变，既遵循学术规范，又自有学者个性，其价值并不会因年月的流逝而减退。

近年，在全球信息化大潮的推动下，我国的计算机产业发展迅猛，对专业人才的需求日益迫切。这对计算机教育界和出版界都既是机遇，也是挑战；而专业教材的建设在教育战略上显得举足轻重。在我国信息技术发展时间较短的现状下，美国等发达国家在其计算机科学发展的几十年间积淀和发展的经典教材仍有许多值得借鉴之处。因此，引进一批国外优秀计算机教材将对我国计算机教育事业的发展起到积极的推动作用，也是与世界接轨、建设真正的世界一流大学的必由之路。

机械工业出版社华章公司较早意识到“出版要为教育服务”。自1998年开始，我们就将工作重点放在了遴选、移译国外优秀教材上。经过多年的不懈努力，我们与Pearson, McGraw-Hill, Elsevier, MIT, John Wiley & Sons, Cengage等世界著名出版公司建立了良好的合作关系，从他们现有的数百种教材中甄选出Andrew S. Tanenbaum, Bjarne Stroustrup, Brian W. Kernighan, Dennis Ritchie, Jim Gray, Alfred V. Aho, John E. Hopcroft, Jeffrey D. Ullman, Abraham Silberschatz, William Stallings, Donald E. Knuth, John L. Hennessy, Larry L. Peterson等大师名家的一批经典作品，以“计算机科学丛书”为总称出版，供读者学习、研究及珍藏。大理石纹理的封面，也正体现了这套丛书的品位和格调。

“计算机科学丛书”的出版工作得到了国内外学者的鼎力襄助，国内的专家不仅提供了中肯的选题指导，还不辞劳苦地担任了翻译和审校的工作；而原书的作者也相当关注其作品在中国的传播，有的还专程为其书的中译本作序。迄今，“计算机科学丛书”已经出版了近两百个品种，这些书籍在读者中树立了良好的口碑，并被许多高校采用为正式教材和参考书籍。其影印版“经典原版书库”作为姊妹篇也被越来越多实施双语教学的学校所采用。

权威的作者、经典的教材、一流的译者、严格的审校、精细的编辑，这些因素使我们的图书有了质量的保证。随着计算机科学与技术专业学科建设的不断完善和教材改革的逐渐深化，教育界对国外计算机教材的需求和应用都将步入一个新的阶段，我们的目标是尽善尽美，而反馈的意见正是我们达到这一终极目标的重要帮助。华章公司欢迎老师和读者对我们的工作提出建议或给予指正，我们的联系方式如下：

华章网站：[www.hzbook.com](http://www.hzbook.com)

电子邮件：[hzjsj@hzbook.com](mailto:hzjsj@hzbook.com)

联系电话：(010)88379604

联系地址：北京市西城区百万庄南街1号

邮政编码：100037



华章科技图书出版中心

## 译者序

Introduction to Embedded Systems—A Cyber-Physical Systems Approach

不同于大多数嵌入式系统的书籍着重于计算机技术在嵌入式系统中的应用，本书的重点是论述系统模型与系统实现的关系，以及软件和硬件与物理环境的相互作用。本书是业界第一本关于 CPS (Cyber-Physical System, 信息物理系统) 的专著。

原书作者美国加州大学伯克利分校 Edward Ashford Lee 教授是世界上嵌入式系统领域的著名学者，也是 CPS 研究的倡导者和引领者之一。Lee 教授领导的团队还开发了一个叫做 Ptolemy II 的系统，这是一个非常优秀的开源嵌入式系统研究与开发平台，可惜国内了解的人不多。

CPS 将计算、网络 and 物理过程集成在一起，CPS 的建模、设计和分析成为本书的重点。

从 CPS 的视角，围绕系统的建模、设计和分析这三个方面，本书分成四大部分。第一部分包括第 2~6 章，分别讲述动态建模、离散建模和混合建模，以及状态机的并发组合与并发计算模型。第二部分包括第 7~11 章，强调嵌入式系统中处理器、存储器架构、输入和输出、多任务处理和实时调度的算法与设计，以及这些设计在 CPS 中的主要作用。第三部分包括第 12~15 章，重点介绍一些系统特性的精确规格、规格之间的比较方法、规格与产品设计的分析方法以及嵌入式软件特性的定量分析方法。第四部分包括两个附录，提供了一些数学和计算机科学的背景知识，有助于加深对文中所介绍知识的理解(可以登录网站 <http://LeeSeshia.org> 获取本书的英文版和更多信息)。

李仁发教授负责本书的整体翻译工作。李实英博士主译了前言、第 1 章以及第一部分和第二部分，贺蓉博士主译了第三部分和第四部分。此外，参与本书翻译工作的还有谢勇、黄鑫、杜家宜、王震、高楠、宋倩、刘琳、周权、吴武飞、谷连军、何翔、许文龙、吴文康、黄晶、李志灿和胡环。

把一种语言表达转换成另一种是一件困难的事情。看似很直白的一个词，虽然理解其词义，但要换一种语言表达时往往煞费苦心，况且 CPS 的研究在国内尚处于起步阶段，许多名词还很难给出确切的中文译名。译者力求忠实地表达书中所介绍的技术，保持原作者的行文风格。此外，本书的翻译以 Leesheshia\_DigitalV1\_03 为基础，并根据作者更新的版本(Leesheshia\_DigitalV1\_05)进行了校对，修正了原稿中的一些改动。限于时间(急于想把此书介绍给国内读者)以及译者水平和经验的不足，译文中难免存在许多不当之处，恳请读者提出宝贵的意见。

本书在翻译过程中得到了 Edward Ashford Lee 教授本人的直接帮助，同时还得到湖南大学嵌入式系统与网络实验室同仁及机械工业出版社许多人士的帮助。对此，译者深表感谢。

## 关于此书

计算机和软件最显著的用途是处理人们所使用的信息。我们用计算机和软件写教材(如本书)、在网上搜索信息、通过电子邮件进行通信以及跟踪财务数据。然而,绝大多数应用中的计算机并非如此显而易见。这些计算机运行于汽车上的引擎、刹车、安全带、安全气囊和音响系统中。它们将声音进行数字编码并转换成无线电信号,然后从手机发送到基站。它们控制微波炉、冰箱和洗碗机。它们运行各种打印机,从台式喷墨打印机到大型工业用的高容量打印机。它们指挥车间里的机器人、发电厂的电力生产、化工厂的各种工序,以及城市的交通灯。它们在生物样本中搜寻细菌,构建人体内部图像,以及测量生命体征。它们处理来自太空的无线电信号,寻找超新星和外星智慧生物。它们给人类生活带来各种玩具,而且让这些玩具能对人的触摸和声音有所反应。它们控制飞机和火车。这些不显眼的计算机称为**嵌入式系统(embedded system)**,而在嵌入式系统上运行的软件称为**嵌入式软件(embedded software)**。

尽管嵌入式系统得到如此广泛的应用,但是从计算机科学相对短暂的发展历史来看,它主要致力于信息处理。直到最近,嵌入式系统才受到研究人员的更多关注。研究界认识到,设计和分析嵌入式系统所需的工程技术是与通用计算机系统不同的。尽管嵌入式系统从20世纪70年代就开始应用,但是长期以来,这些系统被简单地视做小型计算机。最主要的工程问题被理解为如何运用有限资源(有限的处理能力、有限的电源、小型存储器等)的问题。这样一来,工程挑战成为优化设计。由于所有设计都得益于优化,所以该学科与计算机科学的其他方面相比并没有什么独特的地方。它只在应用同样的优化方法方面必须更加积极投入。

目前,研究界已经认识到,嵌入式系统中最主要的挑战来自于它们与物理过程的相互作用,而不是它们的有限资源。“Cyber-Physical System”(CPS)这个术语由美国国家科学基金会的Helen Gill提出,用于描述计算与物理过程的整合。在CPS中,嵌入式的计算机和网络通常采用反馈回路来监视和控制物理过程,在反馈回路中物理过程和计算相互影响。因而,这种系统设计需要理解计算机、软件、网络 and 物理过程的动态融合。正是对于动态融合(joint dynamics)的研究将这个学科与其他学科分离开来。

在研究CPS的过程中,会遇到一些在通用计算中很少出现的关键问题。例如,在通用软件中,执行一个任务的时间与性能相关,但不与正确性相关。执行一个任务耗费更长时间并不是不正确的,只是不太方便,因而不那么有价值。但是在CPS中,执行一个任务所需的时间可能对系统的正确功能实现至关重要。与信息世界相反,在物理世界中时间的流逝是必然的。

而且,在CPS中许多事情会同时发生。物理过程是许多同时发生事情的组合,这与深深植根于顺序步骤中的软件进程不同。Abelson and Sussman(1996)将计算机科学描述为“程序化认识论”(procedural epistemology),即知识贯穿于整个程序。相对而言,在物理世界中,过程很少是程序化的。物理过程是许多并行过程的组合。通过调和影响这些过程的行为来对这些过程的动态状况进行测量和控制是嵌入式系统的主要任务。因此,并发性是CPS固有的。在设计和分析嵌入式软件时,许多技术挑战源于必须建立原本顺序的语义与本质上并发的物理世界之间的桥梁。

## 写作目的

当今,使计算机与物理过程协作需要复杂的技术和底层的设计。因此嵌入式软件设计人员不得与中断控制器、存储器架构、汇编级编程(以开发专用指令或进行精确的时间控制)、设备驱动程序设

计、网络接口和调度策略打交道，而不是只关注所要实现的行为。这些技术的高质量 and 复杂性需要我们重点掌握一些入门知识。这些知识着重于如何对软件、网络 and 物理过程的动态融合进行建模 and 设计，介绍实现这些动态融合的最新（而不是过去的）方法。本书正是力求成为这样一本教材。

大多数关于嵌入式系统的书籍侧重于使计算机与物理系统交互所需的技术 (Barr and Massa, 2006; Berger, 2002; Burns and Wellings, 2001; Kamal, 2008; Noergaard, 2005; Parab et al., 2007; Simon, 2006; Valvano, 2007; Wolf, 2000)。其他书籍则着重于将计算机技术（如编程语言、操作系统、网络等）应用于嵌入式系统中的技术问题 (Buttazzo, 2005a; Edwards, 2000; Pottie and Kaiser, 2005)。虽然这些实现技术可以满足设计人员使嵌入式系统正常工作的需求，但并不是构成该学科的智能核心。其智能核心是将计算与物理动态结合在一起的模型 and 抽象化。

有些教科书在该方向上付出了很多努力。Jantsch (2003) 着重于并发计算模型，Marwedel (2003) 侧重于软件和硬件的行为模式，Sriram and Bhattacharyya (2009) 关注信号处理行为的数据流模型及其在可编程 DSP 上的映射。以上都是非常良好的开端。并发模型（如数据流）以及软件的抽象模型（如状态图）可以作为比命令式编程语言（如 C）、中断和线程以及设计人员必须解决的架构问题（比如缓存）更好的开端。然而，这些教材并不适合入门课程。它们要么太专业要么太高级，或者两者兼而有之。本书力求成为一本入门式的教材，侧重于介绍系统模型及其与系统实现之间的关系。

本书的主题是关于系统模型以及它们与系统实现的关系。这些模型主要是动态的，即系统状态随时间演化。我们不介绍关于组成系统静态信息的结构模型，尽管这些模型对于嵌入式系统设计也非常重要。

应用模型有一个很大的优势。模型具有形式上的特性。可以利用模型定义事件。例如，可以断言一个模型是确定性的 (deterministic)，也就是说，给定相同的输入，它总是产生相同的输出。系统的任何物理实现都不能进行这样绝对的断言。如果模型是对一个物理系统的良好抽象（在此，“良好抽象”是指它只忽略了一些无关紧要的细节），那么模型的决定性断言可以增加系统物理实现的可靠性。这种可靠性是极具价值的，尤其对于嵌入式系统而言，其出现的故障可能危及人的生命。通过学习系统模型可以了解这些系统在物理世界中是如何运行的。

我们的重点是软件和硬件与所处物理环境的相互作用。这就要求对软件和网络的时序动态进行明确建模并精确描述应用中固有的并发特性。事实上，实现技术还不能满足这一要求，这当然不能成为我们传授错误的工程方法的理由，而应该按照设计和建模应有的形式进行教学，并以如何通过最新技术（部分）实现这些目标的关键表述来丰富其内容。因此，当今的嵌入式系统技术，不应该如上述许多教材那样，被描述成一大堆事实和技巧，而应该脚踏实地逐步实现合理的设计实践，重点应该放在什么是合理的设计实践，以及如今的技术如何阻碍和实现它。

Stankovic et al. (2005) 支持这一观点，指出“现有的 RTES (实时嵌入式系统) 设计技术并不能有效地支持可靠而鲁棒的嵌入式系统开发”。他们提出需要“提高编程抽象化的水平”。我们认为，只提高抽象化水平是不够的，应该从根本上改变所使用的抽象。例如，如果没有建立于其上的底层抽象，软件的时间特性则不能有效地利用更高层次的抽象。

我们需要采用鲁棒性好、可预见且具有可重复时序动态的设计 (Lee, 2009a)，这必须通过建立能适当反映信息物理系统实际情况的抽象来实现。其结果将使 CPS 的设计更加复杂，包括更多的自适应控制逻辑、时间演化，以及安全性和可靠性的提高，而避免当今设计的不稳定，很小的改变都成为大问题。

除了处理时序动态，CPS 设计总是面临并发问题的挑战。由于软件是植根于顺序抽象，并发机制（如中断和多任务处理，利用信号和互斥锁）变得非常重要。因此，我们在本书中特别强调对线程、消息传递、死锁避免、竞争条件和数据决策等关键内容的理解。

## 本书未尽事宜

本书的版本并不完整。实际上，要完全涵盖 CPS 意义上的嵌入式系统是不可能的。其中涵盖了

Berkeley(伯克利)本科嵌入式系统课程(<http://LeeSeshia.org>)的具体内容,并希望在将来的版本中包括:传感器和制动器、网络、容错、安全性、仿真技术、控制系统以及硬件/软件协同设计。

## 如何使用本书

本书分为三个主要部分,分别关注建模、设计和分析,如图1所示。这三个部分彼此相对独立,在很大程度上是为了能同时学习。利用虚线框出的七个分区可以实现系统性的学习。每个分区包括两章,假设每个分区需要花两周时间,并且绪论和结束部分各花一周时间,则本书可以在15周内学完。

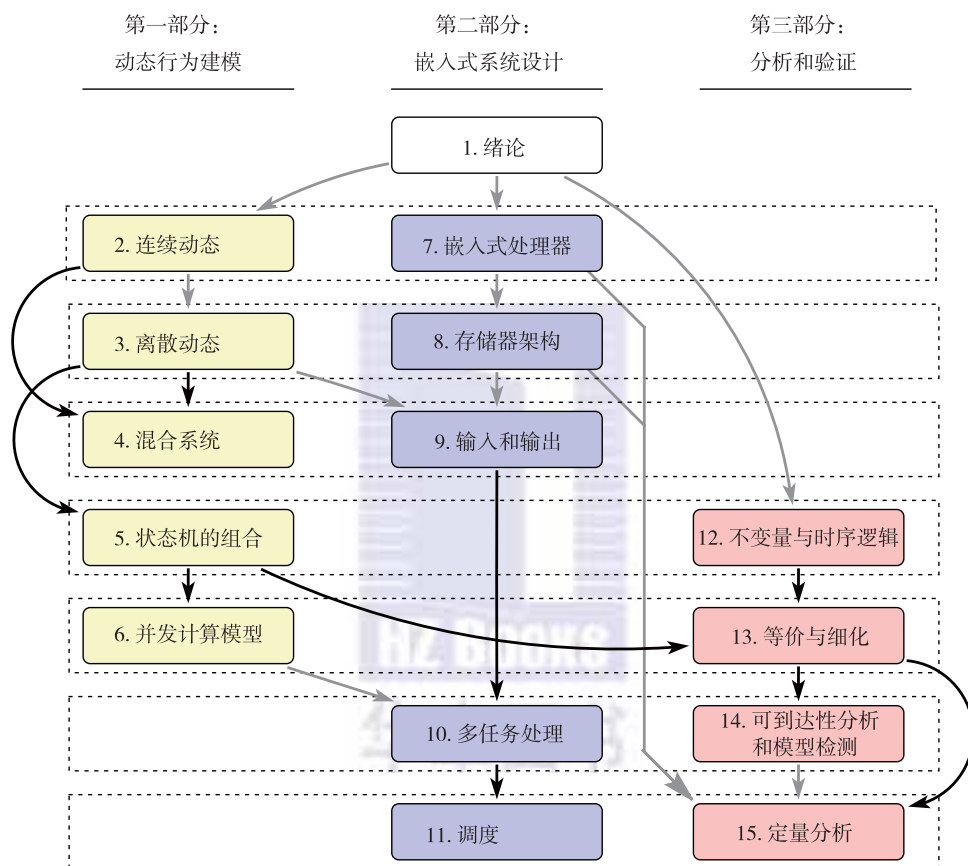


图1 本书各章节之间依赖性强弱分布图。黑色箭头表示章节之间有强依赖关系,灰色箭头表示章节之间有弱依赖关系。如果章节 $i$ 和章节 $j$ 之间有弱依赖关系,则可以只学习章节 $j$ ,而不学习章节 $i$ ,或者略去一些实例或具体的分析方法

附录提供的背景资料在其他教材中也很详尽,但是收入本书对学习可以提供很大帮助。附录A介绍的是集合和函数的概念,这些概念可以提高嵌入式系统学习中常见的精确度。附录B介绍的是复杂度和可计算性理论的基本成果,这些可以加深理解系统建模和分析中的问题。值得注意的是,附录B依赖于第3章讲述的状态机理论,因此应该在读完第3章之后进行学习。

由于技术的最新发展从根本上改变了出版业的技术,本书英文原版以非传统方式出版。至少当前的版本可以以PDF文件方式在线阅读,可从网站<http://LeeSeshia.org>上免费获取。布局适合于中型屏幕,尤其是笔记本电脑、iPad以及其他平板电脑。此外,大量使用超链接和色彩以加强在线阅读体验。

我们力图使英文书适用于电子书形式,理论上可以在不同大小的屏幕上阅读,从而充分利用现有的显示屏。然而,类似于HTML文档,电子书的格式采用回流技术,使页面布局即时重新绘制。其结

果非常依赖于屏幕的大小，在很多屏幕上显得有些怪异，不是最优的显示。因此，我们选择性地对布局进行了控制，不建议在 iPhone 上阅读。

## 读者对象

本书适用于高年级本科生或低年级研究生，以及想要了解嵌入式系统工程原理的实践工程师和计算机专家。读者需要懂得计算机结构(如知道什么是 ALU)、计算机编程(本书采用 C 语言)、基本的离散数学和算法知识，最好对信号和系统也有所了解(比如如何对连续信号进行采样)。

## 致谢

感谢以下人员在过去三年中对本书的贡献和提出的宝贵建议：Murat Arcak, Janette Cardoso, Gage Eads, Stephen Edwards, Suhaib Fahmy, Shanna-Shaye Forbes, Jeff C. Jensen, Jonathan Kotker, Wenchao Li, Isaac Liu, Slobodan Matic, Le Ngoc Minh, Steve Neuendorffer, Minxue Pan, Hiren Patel, Jan Reineke, Rhonda Righter, Chris Shaver, Stavros Tripakis, Pravin Varaiya, Maarten Wiggers, Qi Zhu, 以及 UC Berkeley 的 EECS 149 班的学生，尤其是 Ned Bass 和 Dan Lynch。特别感谢 Elaine Cheong 认真审阅了大多数章节并提出了有益的建议。特别感谢家人的忍耐和支持，尤其是 Helen、Katalina、Rhonda (来自 Edward)，以及 Appa、Amma、Ashwin 和 Bharathi (来自 Sanjit)。

## 错误反馈

如果发现本书中的错误或印刷错误，或者如果有改进或其他建议，请发送电子邮件到 [authors@leeseshia.org](mailto:authors@leeseshia.org)。

请注明该书的版本号和相关的页码，无论是电子版或纸质版。非常感谢！

## 扩展阅读

最近几年出现了很多嵌入式系统方面的教材。这些教材以各种不同的方式介绍这一主题，往往反映一些与嵌入式系统紧密结合的成熟领域的发展前景，如 VLSI 设计、控制系统、信号处理、机器人、实时系统或软件工程。其中有些教材可以作为本书的有益补充，强烈推荐给那些想要深入理解嵌入式系统的读者。

具体而言，Patterson and Hennessy (1996) 尽管不是以嵌入式处理器作为重点，但它是一本计算机架构的经典教材，是对嵌入式处理器架构感兴趣的读者必读的。Sriram and Bhattacharyya (2009) 侧重于信号处理应用，如无线通信和数字媒体，并全面介绍了数据流编程方法。Wolf (2000) 对硬件设计技术和微处理器架构以及它们对嵌入式软件设计的影响给出了精彩的概述。Mishra and Dutt (2005) 介绍了基于结构描述语言(ADL)的嵌入式架构。Oshana (2006) 专门介绍了 Texas Instruments(德州仪器)公司(TI)的 DSP 处理器，综述了架构方法和汇编级编程。

在软件上，Buttazzo(2005a)对实时软件的调度技术进行了全面综述。Liu (2000)对处理软件中的突发实时事件提供了一个目前为止最好的方法。Edwards (2000)介绍了一种用于嵌入式系统设计的域专用高级编程语言。Pottie and Kaiser (2005) 综合介绍了网络技术，特别是嵌入式系统中的无线技术。Koopman (2010)侧重于嵌入式软件的设计过程，包括需求管理、项目管理、测试计划和安全计划。

没有任何一本教材可以全面涵盖所有技术，提供给嵌入式系统工程师。很多教材提供了关于当前设计技术的有益信息(Barr and Massa, 2006; Berger, 2002; Burns and Wellings, 2001; Gajski et al., 2009; Kamal, 2008; Noergaard, 2005; Parab et al., 2007; Simon, 2006)。

## 致教师

在 Berkeley，本书用于高年级本科生课程“嵌入式系统导论”。通过本书的网站主页 <http://LeeSeshia.org> 可以获得大量讲座和实验的资料。另外，还可以获得解题手册。请联系 [authors@leeseshia.org](mailto:authors@leeseshia.org)。

|                                                                              |                    |
|------------------------------------------------------------------------------|--------------------|
| $x _I \leq \tau$                                                             | 时间约束               |
| $\neg$                                                                       | 否定                 |
| $\wedge$                                                                     | 逻辑与                |
| $\vee$                                                                       | 逻辑或                |
| $L(M)$                                                                       | 语言                 |
| $:=$                                                                         | 赋值                 |
| $V_{CC}$                                                                     | 电源电压               |
| $\implies$                                                                   | 蕴涵                 |
| $\mathbf{G}\phi$                                                             | 全局                 |
| $\mathbf{F}\phi$                                                             | 最终                 |
| $\mathbf{U}\phi$                                                             | 直到                 |
| $\mathbf{X}\phi$                                                             | 下一个状态              |
| $L_a(M)$                                                                     | FSM可接受语言           |
| $\lambda$                                                                    | 空序列                |
| $\mathbb{B} = \{0, 1\}$                                                      | 二进制数               |
| $\mathbb{N} = \{0, 1, 2, \dots\}$                                            | 自然数                |
| $\mathbb{Z} = \{\dots, -1, 0, 1, 2, \dots\}$                                 | 整数                 |
| $\mathbb{R}$                                                                 | 实数                 |
| $\mathbb{R}_+$                                                               | 非负实数               |
| $A \subseteq B$                                                              | 子集                 |
| $2^A$                                                                        | 幂集                 |
| $\emptyset$                                                                  | 空集                 |
| $A \setminus B$                                                              | 差集                 |
| $A \times B$                                                                 | 笛卡儿积               |
| $(a, b) \in A \times B$                                                      | 元组                 |
| $A^0$                                                                        | 单元素集               |
| $f: A \rightarrow B$                                                         | 函数                 |
| $f: A \rightharpoonup B$                                                     | 部分函数               |
| $g \circ f$                                                                  | 复合函数               |
| $f^n: A \rightarrow A$                                                       | 函数到幂               |
| $f^0(a)$                                                                     | 恒等函数               |
| $\hat{f}: 2^A \rightarrow 2^B$                                               | 象函数                |
| $(A \rightarrow B)$                                                          | $A$ 到 $B$ 的所有函数的集合 |
| $B^A$                                                                        | $A$ 到 $B$ 的所有函数的集合 |
| $\pi_I$                                                                      | 投影                 |
| $\hat{\pi}_I$                                                                | 提升版投影              |
| $f _C$                                                                       | 限制                 |
| $A^*$                                                                        | 有限序列               |
| $A^{\mathbb{N}}$                                                             | 无限序列               |
| $\omega = \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}, \dots\}$ | 冯·诺依曼数             |
| $A^\omega$                                                                   | 无限序列               |
| $A^{**}$                                                                     | 有限和无限序列            |
| $\square$                                                                    | 空格                 |

# 目 录

Introduction to Embedded Systems—A Cyber-Physical Systems Approach

|               |    |                |    |
|---------------|----|----------------|----|
| 出版者的话         |    | 3.3 有限状态机      | 25 |
| 译者序           |    | 3.3.1 转移       | 25 |
| 前言            |    | 3.3.2 发生响应时    | 27 |
| 符号            |    | 3.3.3 升级函数     | 29 |
|               |    | 3.3.4 确定性和可接受性 | 30 |
| 第1章 绪论        | 1  | 3.4 扩展状态机      | 30 |
| 1.1 应用        | 1  | 3.5 非确定性       | 33 |
| 1.2 一个实例      | 3  | 3.5.1 形式化模型    | 34 |
| 1.3 设计过程      | 4  | 3.5.2 非确定性的用途  | 35 |
| 1.3.1 建模      | 6  | 3.6 行为和轨迹      | 36 |
| 1.3.2 设计      | 6  | 3.7 小结         | 37 |
| 1.3.3 分析      | 7  | 练习             | 38 |
| 1.4 小结        | 8  | 第4章 混合系统       | 41 |
|               |    | 4.1 模态模型       | 41 |
| 第一部分 动态行为建模   |    | 4.1.1 状态机的参量模型 | 41 |
| 第2章 连续动态      | 10 | 4.1.2 连续输入     | 41 |
| 2.1 牛顿力学      | 10 | 4.1.3 状态精化     | 42 |
| 2.2 参量模型      | 13 | 4.2 混合系统的分类    | 43 |
| 2.3 系统的特性     | 15 | 4.2.1 时间自动机    | 43 |
| 2.3.1 因果关系系统  | 15 | 4.2.2 高阶动态     | 45 |
| 2.3.2 无记忆系统   | 16 | 4.2.3 管理控制     | 49 |
| 2.3.3 线性和时不变性 | 16 | 4.3 小结         | 52 |
| 2.3.4 稳定性     | 17 | 练习             | 52 |
| 2.4 反馈控制      | 17 | 第5章 状态机的组合     | 55 |
| 2.5 小结        | 20 | 5.1 并发组合       | 56 |
| 练习            | 20 | 5.1.1 并列同步组合   | 56 |
| 第3章 离散动态      | 22 | 5.1.2 并列异步组合   | 57 |
| 3.1 离散系统      | 22 | 5.1.3 共享变量     | 58 |
| 3.2 状态的概念     | 25 | 5.1.4 级联组合     | 60 |

|                              |     |                                |     |
|------------------------------|-----|--------------------------------|-----|
| 5.1.5 通用组合 .....             | 62  | 7.3 小结 .....                   | 104 |
| 5.2 分层状态机 .....              | 62  | 练习 .....                       | 104 |
| 5.3 小结 .....                 | 65  |                                |     |
| 练习 .....                     | 65  | <b>第 8 章 存储器架构</b> .....       | 106 |
| <b>第 6 章 并发计算模型</b> .....    | 67  | 8.1 存储技术 .....                 | 106 |
| 6.1 模型结构 .....               | 67  | 8.1.1 RAM .....                | 106 |
| 6.2 同步响应模型 .....             | 69  | 8.1.2 非易失性存储器 .....            | 107 |
| 6.2.1 反馈模型 .....             | 70  | 8.2 存储器层次结构 .....              | 107 |
| 6.2.2 形式规范和形式不规范<br>模型 ..... | 71  | 8.2.1 存储映射 .....               | 108 |
| 6.2.3 构建一个固定点 .....          | 72  | 8.2.2 寄存器文件 .....              | 109 |
| 6.3 数据流计算模型 .....            | 73  | 8.2.3 便签式存储器和高速缓冲<br>存储器 ..... | 110 |
| 6.3.1 数据流原理 .....            | 73  | 8.3 存储模型 .....                 | 112 |
| 6.3.2 同步数据流 .....            | 75  | 8.3.1 存储地址 .....               | 112 |
| 6.3.3 动态数据流 .....            | 78  | 8.3.2 栈 .....                  | 113 |
| 6.3.4 结构化数据流 .....           | 79  | 8.3.3 存储器保护单元 .....            | 114 |
| 6.3.5 进程网络 .....             | 79  | 8.3.4 动态存储分配 .....             | 114 |
| 6.4 实时计算模型 .....             | 80  | 8.3.5 C 的存储模型 .....            | 114 |
| 6.4.1 时间触发模型 .....           | 81  | 8.4 小结 .....                   | 115 |
| 6.4.2 离散事件系统 .....           | 83  | 练习 .....                       | 115 |
| 6.4.3 连续时间系统 .....           | 84  | <b>第 9 章 输入和输出</b> .....       | 117 |
| 6.5 小结 .....                 | 86  | 9.1 I/O 硬件 .....               | 117 |
| 练习 .....                     | 86  | 9.1.1 脉宽调制 .....               | 117 |
| <b>第二部分 嵌入式系统设计</b>          |     | 9.1.2 通用数字 I/O .....           | 118 |
| <b>第 7 章 嵌入式处理器</b> .....    | 90  | 9.1.3 串行接口 .....               | 120 |
| 7.1 处理器类型 .....              | 90  | 9.1.4 并行接口 .....               | 122 |
| 7.1.1 微控制器 .....             | 90  | 9.1.5 总线 .....                 | 123 |
| 7.1.2 DSP 处理器 .....          | 91  | 9.2 并发环境下的顺序软件 .....           | 123 |
| 7.1.3 图形处理器 .....            | 95  | 9.2.1 中断和异常 .....              | 123 |
| 7.2 并行处理 .....               | 95  | 9.2.2 原子性 .....                | 125 |
| 7.2.1 并行处理与并发处理 .....        | 95  | 9.2.3 中断控制器 .....              | 126 |
| 7.2.2 流水线 .....              | 97  | 9.2.4 中断建模 .....               | 126 |
| 7.2.3 指令级并行 .....            | 99  | 9.3 模拟/数字接口 .....              | 129 |
| 7.2.4 多核架构 .....             | 102 | 9.3.1 数模转换和模数转换 .....          | 129 |
|                              |     | 9.3.2 信号调节 .....               | 130 |

|                              |     |                                     |     |
|------------------------------|-----|-------------------------------------|-----|
| 9.3.3 采样和走样 .....            | 132 | 12.2 线性时序逻辑 .....                   | 173 |
| 9.4 小结 .....                 | 134 | 12.2.1 命题逻辑公式 .....                 | 174 |
| 练习 .....                     | 134 | 12.2.2 LTL 公式 .....                 | 175 |
| <b>第 10 章 多任务处理</b> .....    | 138 | 12.2.3 LTL 公式的应用 .....              | 177 |
| 10.1 命令式程序 .....             | 139 | 12.3 小结 .....                       | 178 |
| 10.2 多线程 .....               | 141 | 练习 .....                            | 179 |
| 10.2.1 创建线程 .....            | 142 | <b>第 13 章 等价与精化</b> .....           | 181 |
| 10.2.2 实现多线程 .....           | 143 | 13.1 规格建模 .....                     | 181 |
| 10.2.3 互斥 .....              | 144 | 13.2 类型等价与类型精化 .....                | 182 |
| 10.2.4 死锁 .....              | 146 | 13.3 语言等价与包含 .....                  | 183 |
| 10.2.5 存储一致性模型 .....         | 147 | 13.4 模拟 .....                       | 187 |
| 10.2.6 多线程问题 .....           | 148 | 13.4.1 模拟关系 .....                   | 188 |
| 10.3 进程和消息传递 .....           | 149 | 13.4.2 形式化模型 .....                  | 189 |
| 10.4 小结 .....                | 152 | 13.4.3 传递性 .....                    | 190 |
| 练习 .....                     | 152 | 13.4.4 模拟关系的非<br>唯一性 .....          | 190 |
| <b>第 11 章 调度</b> .....       | 154 | 13.4.5 模拟与语言包含 .....                | 191 |
| 11.1 调度的基础知识 .....           | 154 | 13.5 互模拟 .....                      | 191 |
| 11.1.1 调度决策 .....            | 154 | 13.6 小结 .....                       | 193 |
| 11.1.2 任务模型 .....            | 155 | 练习 .....                            | 193 |
| 11.1.3 调度程序的比较 .....         | 157 | <b>第 14 章 可达性分析和<br/>模型检测</b> ..... | 196 |
| 11.1.4 调度程序的实现 .....         | 157 | 14.1 开放式与封闭式系统 .....                | 196 |
| 11.2 单调速率调度 .....            | 158 | 14.2 可达性分析 .....                    | 197 |
| 11.3 最早时限优先 .....            | 160 | 14.2.1 $Gp$ 验证 .....                | 197 |
| 11.4 调度和互斥 .....             | 163 | 14.2.2 显态模型检测 .....                 | 198 |
| 11.4.1 优先级倒置 .....           | 163 | 14.2.3 符号化模型检测 .....                | 199 |
| 11.4.2 优先级继承协议 .....         | 163 | 14.3 模型检测中的抽象 .....                 | 201 |
| 11.4.3 优先级上限协议 .....         | 164 | 14.4 活跃属性的模型检测 .....                | 203 |
| 11.5 多处理器调度 .....            | 166 | 14.4.1 属性的自动机<br>表达 .....           | 203 |
| 11.6 小结 .....                | 168 | 14.4.2 寻找可接受循环 .....                | 205 |
| 练习 .....                     | 169 | 14.5 小结 .....                       | 206 |
| <b>第三部分 分析和验证</b>            |     | 练习 .....                            | 207 |
| <b>第 12 章 不变量与时序逻辑</b> ..... | 172 |                                     |     |
| 12.1 不变量 .....               | 172 |                                     |     |

|                          |     |                         |     |
|--------------------------|-----|-------------------------|-----|
| <b>第 15 章 定量分析</b> ..... | 208 | 15.4.1 最优化问题的形式化        | 216 |
| 15.1 关注的问题 .....         | 208 | 15.4.2 逻辑流约束            | 218 |
| 15.1.1 极限分析 .....        | 208 | 15.4.3 基本块的界限           | 220 |
| 15.1.2 阈值分析 .....        | 209 | 15.5 其他定量分析问题 .....     | 221 |
| 15.1.3 一般情况分析 .....      | 209 | 15.5.1 存储界限分析           | 221 |
| 15.2 程序图 .....           | 209 | 15.5.2 能耗和功耗分析          | 222 |
| 15.2.1 基本块 .....         | 210 | 15.6 小结 .....           | 223 |
| 15.2.2 控制流图 .....        | 210 | 练习 .....                | 223 |
| 15.2.3 函数调用 .....        | 210 |                         |     |
| 15.3 执行时间的决定因素 .....     | 212 | <b>第四部分 附 录</b>         |     |
| 15.3.1 循环界限 .....        | 212 | <b>附录 A 集合和函数</b> ..... | 226 |
| 15.3.2 指数的路径空间 .....     | 214 | <b>附录 B 复杂度和可计算性</b>    |     |
| 15.3.3 路径的可行性 .....      | 214 | 理论 .....                | 231 |
| 15.3.4 存储层次 .....        | 215 | <b>参考书目</b> .....       | 241 |
| 15.4 执行时间分析的基础 .....     | 216 |                         |     |

## 绪 论

CPS(Cyber-Physical System)是计算与物理过程的整合。嵌入式计算机和网络通过反馈回路监视和控制物理过程,在反馈回路中物理过程与计算相互影响。作为一项智力挑战,CPS是物理层和信息层的交集,而不是并集。物理组件和计算组件不能分开理解。相反,必须理解它们之间的相互作用。

本章通过一些 CPS 应用阐述这种系统的工程原理和设计流程。

### 1.1 应用

CPS 应用不可争辩地具有使 20 世纪 IT 技术革新黯然失色的潜力。考虑以下实例。

**例 1.1:** 心脏手术通常需要停止心脏跳动来实施手术,然后再使心脏重新跳动。这种手术是极其有风险的,而且会带来很多有害的副作用。很多研究团队一直在研究另一种方法——可以在心脏跳动时做手术,而不需要使之停止跳动。有两个重要想法可以使之成为可能。首先,手术工具可以由机器人控制,使它们能跟随心脏的跳动而移动(Kremen, 2008)。因此,外科医生可以在心脏持续跳动的同时使用工具给心脏上的一点施加定量的压力。其次,立体视频系统可以向外科医生展现一个静止心脏的假象视频(Rice, 2008)。对于外科医生而言,心脏看起来停止了跳动,而事实上心脏是在持续跳动的。要实现这样的外科手术系统,需要大量的心脏模型、工具、计算硬件以及软件。它需要精心设计的软件以保证精确的计时和处理故障的安全备用行为,而且还需要对模型和设计进行详细分析来提供高可靠性。

**例 1.2:** 考虑城市中的交通灯和车辆共同配合来保证有效的交通流。特别地,假设只有在实际的十字路口才需要因为红灯而停车。这样的系统可以通过检测路上车辆等昂贵基础设施来实现。但是一种更好的方法是让车辆之间进行协作。它们记录自己的位置,并进行通信以互相配合共享资源,比如十字路口。当然,系统的可靠性是其可行性的关键。故障是灾难性的。

**例 1.3:** 设想一架能够防止坠落的飞机。虽然不能杜绝所有可能导致坠落的因素,但是一个精心设计的飞行控制系统可以防止很多因素。这样的系统是很好的信息物理系统实例。

在传统的飞机中,飞行员通过连接驾驶舱控制器的机械和液压装置以及机翼的可移动水平面和飞机尾部来控制飞机。在电控式(fly-by-wire)飞机中,飞行员发出的指令经由飞行计算机进行调节,并通过网络将电子信号传送到机翼和机尾的制动器。电控式飞机比传统飞机轻很多,因而更加省燃料。而且已证实电控式飞机更加可靠。事实上,所有新型飞机都被设计成电控系统。

在电控式飞机中,由于计算机调节来自飞行员的指令,所以计算机可以修正这些指令。很多现代飞行控制系统都在某些情况下修正飞行员的指令。例如,Airbus 公司制造的商业飞机采用一种称为飞行状态防护(flight envelope protection)的技术防止飞机超出安全工作范围。例如,可以防止飞行员引起飞机失速。

飞行状态防护的概念可以得到扩展来帮助防止一些其他坠落因素。例如, Lee(2001)提出一种软壁(soft wall)系统, 这种系统一旦实现, 就可以对安装该系统的飞机进行跟踪定位, 并防止飞机撞到障碍物上, 如山脉和建筑物。利用 Lee 方案, 当飞机接近障碍物边界时, 电控式飞行控制系统会产生一个虚拟推动力迫使飞机远离障碍物。飞行员感觉飞机好像碰撞到一堵软壁而转移方向。设计和部署这种系统在技术和非技术上还有很多挑战。这些问题的讨论可参见 Lee(2003)。

尽管上例中的软壁系统是非常超前的, 但是已经有一些不那么超前的自动安全系统得到部署或处于最高级研发阶段。例如, 当今很多车辆能够检测无意的车道变化并提醒驾驶员。考虑一些更具挑战性的自动纠正驾驶员行为的问题。显而易见, 这比仅仅提醒驾驶员要困难得多。如何能保证系统在需要时进行响应和接管, 并在一定程度上进行实际干预呢?

显然可以设想很多其他应用, 比如老年人辅助系统, 能让外科医生进行远距离手术的远程外科手术系统, 以及需要在稳定电压下工作的家用电器。此外, 还很容易设想利用 CPS 改进很多现有的系统, 比如机器人制造系统, 发电和配电, 化工厂的过程控制, 分布式计算机游戏, 产品的运输, 建筑物的供暖、制冷和照明, 电梯等客运工具, 以及能够监控自身健康状况的桥梁。这些改进对安全、能源消耗和经济具有极其深远的影响。

以上很多实例可以采用如图 1-1 所示的结构进行部署。图中有三个主要部分。首先, **物理设备**是信息物理系统中的“物理”部分。这只是系统中没有利用计算机或数字网络实现的部分。它包括机械零件、生物或化学过程, 以及人工操作。第二, 有一个或多个**计算平台**, 由传感器、驱动器、一台或多台计算机, 以及(可能的)一个或多个操作系统组成。第三, 有一个**网络结构**, 它提供计算机进行通信的机制。平台和网络结构共同构成信息物理系统的“信息”部分。

如图 1-1 所示的是两个网络连接平台, 每个都有各自的传感器和/或驱动器。驱动器的行为影响传感器从物理设备接收到的数据。图中, 平台 2 通过驱动器 1 控制物理设备。它通过传感器 2 测量物理设备的工作进程。图中标为计算 2 的方框实现一个**反馈控制回路**, 根据传感器数据决定发出什么样的命令到驱动器。这种回路称为**反馈控制回路**。平台 1 利用传感器 1 进行其他测量, 并通过网络结构将消息送给平台 2。计算 3 实现另外一个控制规则, 与计算 2 融合在一起, 可能抢占计算 2。

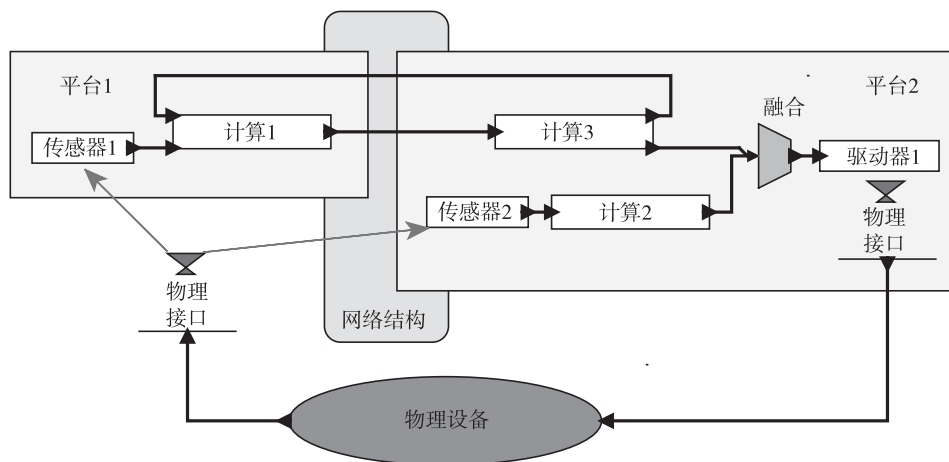


图 1-1 CPS 的结构实例

例 1.4：考虑一个提供按需打印服务的高速打印机。它的结构大概与图 1-1 类似，但是具有更多的平台、传感器和驱动器。驱动器控制电动机推动纸张通过打印机，将墨水印在纸上。控制规则包括纸张拉伸补偿策略，通常取决于纸张类型、温度以及湿度。图 1-1 所示的网络结构也许用于卡纸的情况下快速关机以防止损坏设备。这种关机要求整个系统紧密协调以防止崩溃。类似的情况也可见于高端仪器系统以及能源生产和调度(Eidson et al., 2009)。

### 关于术语 CPS

CPS 这个术语大约出现于 2006 年，由美国国家科学基金会的 Helen Gill 提出。虽然大家都熟悉“信息空间”(cyberspace)这个术语，也许会将它与 CPS 联系起来，但是 CPS 的根源更加久远。认为“信息空间”和 CPS 源于相同的根源——“控制论”(cybernetics)应该是对的，而不应该认为一个衍生于另外一个。

“控制论”这个术语是由 Norbert Wiener (Wiener, 1948)提出的，他是一名对控制系统理论的发展具有深远影响的美国数学家。在二战期间，Wiener 首创了高射炮自动瞄准和射击技术。虽然他采用的机制没有涉及数字计算机，但其中的理论与现今用于各类基于计算机的反馈控制系统的理论相类似。控制论这个术语是 Wiener 从希腊文中的 κυβερνητης (kybernetes) 一词衍生来的，意指舵手、管理者、领航员或方向舵。这个比喻非常适合控制系统。

Wiener 将控制论视为控制和通信的结合。他的控制概念深深植根于闭环反馈，其中控制逻辑是由物理过程的测量值驱动，并反过来驱动物理过程。Wiener 虽然没有使用数字计算机，但控制逻辑实际上是一个计算，因此控制论是物理过程、计算和通信的结合。

Wiener 没有预料到数字计算和网络的巨大影响力。CPS 可以粗略地解释为信息空间与物理过程的结合，这一事实有助于强调 CPS 将有巨大的影响力。CPS 充分发挥了惊人的信息技术，甚至远远超过 Wiener 所处时代最疯狂的梦想。

## 1.2 一个实例

本节主要讲述 CPS 的一个实例，目的是借用这个实例说明本书所述内容的重要性。具体应用如斯坦福大学多代理控制的自主旋翼飞行器实验平台(STARMAC)，由 Claire Tomlin 和同事在斯坦福和伯克利合作开发(Hoffmann et al., 2004)。STARMAC 是一个小型四旋翼飞行器，如图 1-2 所示。它的主要目的是提供一个多飞行器自主控制技术的实验平台。目标是使多个飞行器合作完成一个通常的任务。

使这样的系统正常工作将面临巨大的挑战。第一，飞行器的控制至关重要。主要的驱动装置是四个旋翼，它们产生可变的向下推力。通过平衡来自四个旋翼的推力，飞行器可以起飞、降落、转向，甚至在空中翻转。如何确定需要怎样的推力呢？这就需要复杂的控制算法。

第二，飞行器的重量是另一个重要问题。它越重，就需要携带越多的能量，这无疑又会增加飞行器的重量。飞行器越重就需要越大的推力来飞行，这意味着需要更大更强的马达和旋翼。当飞行器达到一定重量，旋翼会对人类造成威胁，这是设计要跨越的主要门槛。即便

是一个相对较轻的飞行器，安全性也是重点关注的问题，因而系统应该设计有故障处理部件。



图 1-2 飞行中的四旋翼飞行器 STARMAC(该图已获授权)

第三，飞行器需要上下文操作，与环境进行交互。例如，它可能由人通过遥控器进行连续控制，或者可能设为自主操作、起飞、执行任务、返回以及着陆。自主操作是非常复杂和具有挑战性的，因为它没有人的帮助。自主操作需要更加精密的传感器。飞行器需要跟踪自己所在的位置(它需要定位功能)。需要感知障碍物，需要知道哪是地面。如果具有更好的设计，这样的飞行器甚至可以自主降落在颠簸的船甲板上。飞行器还需要不断监测自身的状态，检测故障并做出反应，以化解危机。

不难想象，很多其他应用程序具有与四旋翼飞行器同样的问题。四旋翼飞行器降落在颠簸的船甲板上的问题与在跳动的心脏上进行手术(见例 1.1)的问题类似。它们都需要对动态环境进行详细建模(船，心脏)，还需要对嵌入式系统动态之间(四旋翼飞行器，机器人)的相互作用有清晰的认识。

本章的其余部分将对本书的各个部分进行概述，利用四旋翼飞行器的实例来说明各部分在系统设计中所起的作用。

1.3 设计过程

本书的目的是理解如何设计和实现信息物理系统。图 1-3 显示的是这个过程三个主要部分：建模，设计和分析。建模是通过模拟加深系统理解的过程。模型是对系统的模拟，并反映系统的特性。模型具体说明系统要做什么。设计是创建系统的结构。它指定系统如何完成它的功能。分析是通过剖析进而对系统有更深入理解的过程。它具体说明系统为什么能够完成它的功能(或者为什么无法完成模型认为它能完成的功能)。

从图 1-3 可以看出，这个过程的三部分有重叠，设计过程在这三个部分中反复移动。通常，这个过程是从建模

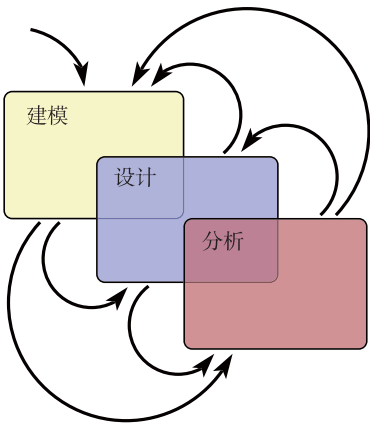


图 1-3 创建嵌入式系统是一个建模、设计和分析的反复过程

开始的，其目的是要理解问题，并制定解决方案。

**例 1.5：**对于 1.2 节的四旋翼飞行器问题，可以从构建模型开始，将人发出的纵向或横向移动命令转换成四个马达产生推力的命令。模型将显示如果四个旋翼上的推力不相同，则飞行器发生倾斜和横向移动。

这种模型可能采用第 2 章介绍的那些方法，建立微分方程来描述飞行器的动力系统。然后采用第 3 章介绍的方法来构建状态机，对起飞、降落、悬停和侧飞等操作模式进行建模。接着采用第 4 章中的方法将两种模型进行融合，建立系统的混合系统模型以研究操作模式之间的转换。第 5 章和第 6 章的方法将提供多飞行器模型、飞行器与其所处环境之间的相互作用模型，以及飞行器内部组件之间的相互作用模型等的组合机制。

这个过程可能很快进展到设计阶段，在此阶段开始选择元件，将它们（电机、电池、传感器、微处理器、存储系统、操作系统、无线网络等）组合到一起。初始原型可能会暴露出模型中的缺陷，因而回到建模阶段并对模型进行修正。

**例 1.6：**图 1-4 显示的是第一代 STARMAC 四旋翼飞行器的硬件结构。图的左边和底部是一些传感器，飞行器通过它们确定身在何处(定位)以及周边环境。中间的两个方框是不同的微处理器。Robostix 是一个 Atmel AVR 8 位微控制器，它没有操作系统，通过一些底层控制算法使飞行器保持飞行。另外两个处理器在操作系统的帮助下执行较高层的任务。这两个处理器都有无线连接，可以用于协调飞行器和地面控制器。

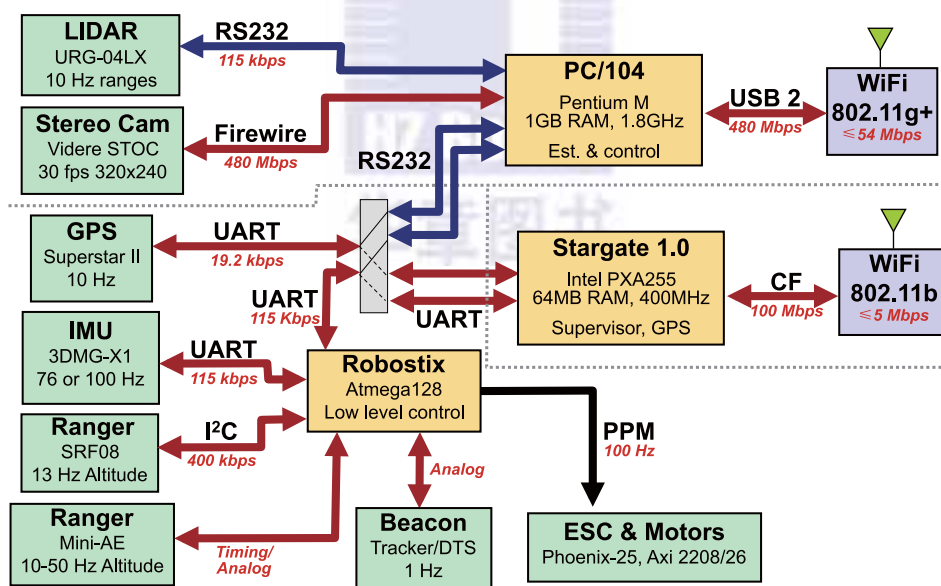


图 1-4 STARMAC 结构(该图已获授权)

第 7 章主要讨论处理器的架构，为比较不同架构各自的优点提供一些基础知识。第 8 章主要讲述存储系统的设计，强调它们对整个系统行为的影响。第 9 章主要介绍处理器与传感器和驱动器之间的接口。第 10 章和第 11 章侧重于软件架构，特别强调如何协调多个实时任务。

在一个良好的设计过程中，数据分析是初期的重要部分。分析应用于模型和设计。例如，对模型的安全状况进行分析以保证“如果飞行器与地面距离不足 1 米，则它的速度不大

于 0.1 米/秒”这一断言的不变性。可能要对软件时间行为的设计进行分析,以确定系统对紧急停机命令做出响应需要多长时间。有些问题的分析将涉及模型和设计两者的细节。例如,对于四旋翼飞行器,如果网络连接中断而无法与飞行器进行通信,系统将如何反应?理解这点是非常重要的。飞行器如何能检测到通信中断?这就需要对网络和软件进行精确建模。

**例 1.7:** 对于四旋翼飞行器问题,采用第 12 章介绍的方法具体说明飞行器运行的关键安全要求。然后采用第 13 章和第 14 章的方法通过软件实现来验证这些安全特性是否达到要求。最后采用第 15 章的方法来确定软件是否满足实时约束。

与图 1-3 所示的设计过程相对应,本书分为三个主要部分,重点介绍建模、设计和分析(见“前言”中的图 1)。接下来讲述这三个部分采用的方法。

### 1.3.1 建模

本书的建模部分主要关注动态行为模型。第 2 章首先简单介绍物理动态建模,然后特别关注时间上的连续动态。接下来第 3 章讲述离散动态,其主要形式是状态机。第 4 章讨论以上两种动态的结合,即混合系统。第 5 章侧重于状态机的并发组合,强调组合语义是设计人员必须解决的重要问题。第 6 章概述并发计算模型,包括实践人员常用的设计工具(如 Simulink 和 LabVIEW)中用到的很多计算模型。

本书的建模部分将**系统**定义为属于同一整体的各个部分的组合。相对于软件和算法这样的概念或**逻辑系统**,**物理系统**是已经被实现的。系统的**动态**(dynamics)是时间上的演化:其状态如何改变。物理系统的**模型**是对要产生的系统特性从某些方面进行的描述。在本书中,模型具有可进行系统分析的数学特性。这个模型对系统特性进行模拟,进而产生对系统的理解。

模型自身就是一个系统。避免将模型和被建模的系统混淆是非常重要的。它们两个是不同的。如果系统的模型是对系统特性的精确描述,那么认为该模型具有**高精度性**。如果忽略细节,它就是对系统的**抽象**。物理系统的模型不可避免地要忽略细节,因此它们通常是对系统的抽象。本书的主要目的是加深理解如何使用模型,以及如何取其所长避其所短。

信息物理系统是将物理子系统与计算和网络融合在一起的系统。信息物理系统的模型通常包括三个部分。这些模型通常需要表达动态和**静态**两种特性(静态特性是指在系统运行的过程中没有变化的特性)。

本书相应部分讲述的每个建模方法都是一个大主题,用一章甚至一本书都介绍不完。事实上,这些模型受到工程、物理、化学和生物学的分支学科关注。我们的方向目标是工程师。假设他们具有数学动态建模的背景知识(给出物理学应用实例的微积分课程已经足够),接下来重点理解如何构造多样化的模型。这样就形成信息物理系统问题的核心,因为将以逻辑和概念为主的信息侧面和涉及具体实现的物理侧面相结合就是该问题的核心。因此文中不打算全面介绍,而是介绍一些工程师广泛使用和熟知的建模技术,并将它们组合构成一个信息物理系统。

### 1.3.2 设计

本书的第二部分与以上有很大不同,主要反映所讨论主题的内在多样性。这部分的重点是嵌入式系统设计,强调它们在 CPS 中的作用。第 7 章讨论处理器架构,强调最适合嵌入式系统的一些特殊性能。第 8 章讲述存储架构,包括编程语言中的存储模型等抽象,存储技术

等物理性能, 存储层次(缓存, 中间结果暂存器等)等架构性能。重点在于存储架构如何影响动态过程。第9章是关于软件世界和物理世界之间的接口, 讨论的是软件和计算机结构中的输入/输出机制, 以及数/模接口, 包括采样。第10章介绍操作系统构成的一些概念, 特别强调多任务处理。重点是关于使用多线程等底层机制的缺陷, 让读者明白本书第一部分所讲建模技术的真正价值。第11章介绍实时调度, 涵盖该领域的很多经典研究成果。

设计部分的所有章节特别关注提供并发性和时间控制的机制, 因为这些问题在信息物理系统的设计过程中尤为突出。部署产品时, 嵌入式处理器通常有指定的功能。它们可以控制汽车发动机或者测量北极冰层的厚度, 而不是通过用户自定义软件执行任意功能。因此, 处理器、存储架构、I/O机制和操作系统都可以更加专业化。专业化可以带来更多的好处。例如, 系统的能量消耗将极大降低, 因此即使采用小电池也可以工作很长时间。或者可能采用专门硬件来执行那些通用硬件需要高代价的操作, 如图像分析。这部分的目的是让读者对众多可用技术产品进行客观的评价。

这部分是要教授学生如何实现系统, 同时思考传统的抽象层以外的东西, 例如, 硬件和软件、计算和物理过程。这种跨层思考对于通常的系统实现非常重要, 在具有异构特性的嵌入式系统中尤其如此。例如, 程序员要实现一个用实数表示的控制算法时, 必须对计算机运算(如定点数)有深刻的理解, 以便建立一个可靠的实现。与之类似, 要实现一个必须满足实时约束的汽车软件, 设计人员必需熟悉处理器的特性, 如流水线和缓存, 这些会影响任务的执行时间从而影响系统的实时行为。同样, 要实现一个中断驱动或多线程软件, 实现人员必须理解底层软硬件平台提供的原子操作, 并使用适当的同步结构以保证其正确性。这部分内容旨在让读者理解这些跨层内容, 并通过练习加深理解, 而不是对不同的实现方法和平台进行长篇累牍的综述。

### 1.3.3 分析

每个系统的设计必须满足特定的要求。通常对安全性有高要求的嵌入式系统, 关键在于保证系统满足要求。这样的系统要求也称为**特性(property)**或**规格(specification)**。从下面引用的句子可以看出对规格的迫切需要, 摘自 Young et al. (1985):

“没有规格的设计无法判断对或错, 只能让人吃惊!”

本书的分析部分着重于系统特性的精确规格、规格比较的方法, 以及规格和最终设计分析的方法。为了强调动态过程, 第12章重点讨论时序逻辑, 对系统动态特性进行精确描述。这些描述被视为模型。第13章侧重于模型之间的关系。一个模型是否是另一个模型的抽象? 它们在某种意义上是等价的吗? 具体来说, 该章将类型系统作为比较模型静态特性的方法, 将语言包含(language containment)和模拟关系(simulation relation)作为比较模型动态特性的方法。第14章主要介绍模型可能呈现的大量动态行为的分析方法, 特别强调模型检测是一种探究这些行为的方法。第15章是关于嵌入式软件特性的定量分析, 如计算程序消耗的最大资源。重点是执行时间分析, 包括能源和存储器使用等特性的定量分析。

在当前的工程实践中, 通常用英语等自然语言说明系统需求。对系统需求进行精确的说明以避免自然语言所固有的歧义是非常重要的。这部分的目的是要用不易出错的形式化方法来取代描述性方法。

重要的是, 形式化规范还可以利用自动化技术对模型和实现两者都进行形式化验证(formal verification)。分析部分向读者介绍形式化验证的基础知识, 包括等价与精化检测, 以及可到达

性分析和模型检测等概念。在讨论这些验证方法的过程中，试图让验证工具的使用者认识到底层的東西，使他们从中获得最大好处。例如，通过实例讨论支持“用户的看法”，如何利用模型检测找出并发软件中难以捉摸的错误，或者如何利用可达性分析计算一个控制策略，使机器人完成特定的任务。

#### 1.4 小结

CPS 本质上是异构混合体。它将计算、通信和物理动态过程融于一体。与同构系统相比，更难以对它进行建模、设计和分析。本章简单介绍了书中用于建模、设计和分析的工程原理。



## 动态行为建模

本书的这部分是讲述嵌入式系统的建模，重点是软件和物理动态的联合建模。第 2 章中讨论物理系统动态，主要是连续行为建模的一些成熟技术。第 3 章讨论离散行为的建模技术，它能够更好地反映软件行为。第 4 章将这两类模型放在一起，考虑如何将离散和连续行为通过混合系统联合建模。第 5 章和第 6 章专门讲述如何协调物理世界固有的并发性和软件世界固有的串行性。第 5 章讨论串行状态机如何进行并发组合，特别引入同步结构的概念。第 6 章将显示同步结构只是实现并发组合的方法之一。

## 连续动态

本章介绍用于物理系统的动态建模技术。首先讲述力学部分，即运动（这个问题被称为**经典力学**）。研究这部分动态的技术已经广泛扩展到其他物理系统，包括电路、化学过程和生物过程。对于大多数人来说，力学比较容易得到直观的认识，因此可以给出更加具体的实例。力学的运动部分常常使用**微分方程**或**积分方程**来建模。这些模型实际上只能对“平滑”运动有效（可以利用线性、时不变性和连续性的概念进行更加精确的描述）。而对于不是平滑的运动，比如力学碰撞的建模，可以利用**模态模型**（modal model），采用不同模式之间的突变转变（概念上的即时性）来表示不同的模式。力学碰撞可以有效地建模成离散和即时的事件。平滑运动和这些离散事件的联合建模问题称为**混合系统建模**，将在第 4 章讨论。这种离散行为和连续行为的组合使我们向信息过程和物理过程的联合建模更靠近了一步。

首先给出一个简单的运动方程，以常微分方程的形式描述一个系统的模型。然后展示这些**常微分方程**在参量模型（actor model）中是如何体现的，其中包括在建模语言中广泛使用的模型类，如 LabVIEW（美国国家仪器公司）和 Simulink（MathWorks 公司）。接下来考虑此类模型的特性，如线性、时不变性和稳定性，并考虑操作模型时上述特性的重要性。我们给出一个简单的反馈控制系统实例，该系统用于使不稳定的系统趋于稳定。这些系统的控制器常常利用软件来实现，因此可以作为信息物理系统中的一个典型案例。整个系统的特性都是依赖于信息部分和物理部分的特性。

### 2.1 牛顿力学

本节将简要回顾一下经典力学的理论。这不过是用于构建一个令人感兴趣的模型，并不全面。有兴趣的读者可以参考一些经典的力学文献，包括 Goldstein(1980)、Landau and Lifshitz(1976)、Marion and Thornton(1995)。

物理对象在空间中的运动可以表示为**六个自由度**（six degrees of freedom），如图 2-1 所示。其中三个表示三维空间中的位置，另外三个表示三维空间中的方向。定义三个轴为  $x$ 、 $y$  和  $z$ ，按照惯例  $x$  轴的延伸方向向右， $y$  轴的延伸方向向上， $z$  轴的延伸方向垂直纸面向外。**横滚角**（roll） $\theta_x$  为绕  $x$  轴的旋转角，按照惯例 0 度表示沿  $z$  轴的水平方向（也就是说，这个角度是相对于  $z$  轴的）。**方位角**（yaw） $\theta_y$  为绕  $y$  轴的旋转角，按照惯例 0 度表示直接指向右方（也就是说，这个角度是相对于  $x$  轴的）。**俯仰角**（pitch） $\theta_z$  为围绕  $z$  轴的旋转角，按照惯例 0 度表示指向水平方向（也就是说，这个角度是相对于  $x$  轴的）。

空间中物体的位置是通过 6 个  $f: \mathbb{R} \rightarrow \mathbb{R}$  形式的函数来表示的，其中定义域表示时间，值域表示沿轴的距离或相对于轴的角度<sup>⊖</sup>，这种形式的函数称为**连续时间信号**（continuous-

⊖ 如果对符号不太熟悉，可参见附录 A。

time signal)<sup>⊖</sup>。这些函数常统称为矢量值函数  $\mathbf{x} : \mathbb{R} \rightarrow \mathbb{R}^3$  和  $\boldsymbol{\theta} : \mathbb{R} \rightarrow \mathbb{R}^3$ ，其中  $\mathbf{x}$  表示位置， $\boldsymbol{\theta}$  表示方向。

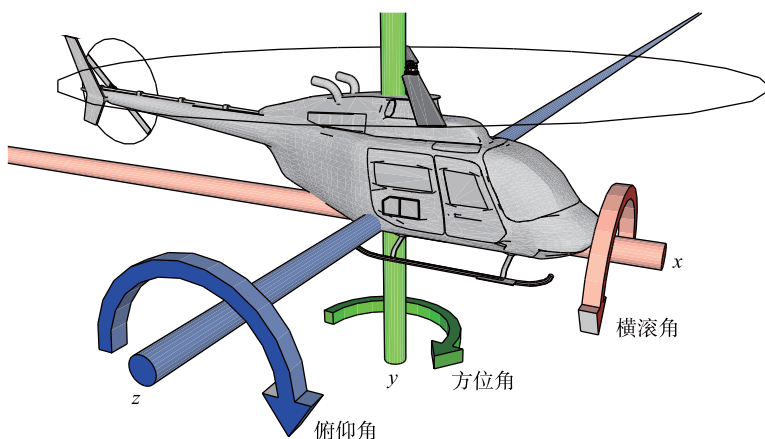


图 2-1 六自由度模型(包括俯仰角、横滚角、方位角)

位置或方向的变化符合牛顿第二定律，与力和加速度有关。加速度是位置的二阶导数。处理位置信息的方程式为

$$\mathbf{F}(t) = M\ddot{\mathbf{x}}(t) \quad (2.1)$$

其中， $\mathbf{F}$  为三个方向上的力矢量， $M$  为物体的质量， $\ddot{\mathbf{x}}$  是  $\mathbf{x}$  对时间的二阶导数(即加速度)。速度是加速度的积分，由下式给出

$$\forall t > 0, \dot{\mathbf{x}}(t) = \dot{\mathbf{x}}(0) + \int_0^t \ddot{\mathbf{x}}(\tau) d\tau$$

其中， $\dot{\mathbf{x}}(0)$  是三个方向上的初速度。通过式(2.1)，上式变为

$$\forall t > 0, \dot{\mathbf{x}}(t) = \dot{\mathbf{x}}(0) + \frac{1}{M} \int_0^t \mathbf{F}(\tau) d\tau$$

位置是速度的积分，

$$\begin{aligned} \mathbf{x}(t) &= \mathbf{x}(0) + \int_0^t \dot{\mathbf{x}}(\tau) d\tau \\ &= \mathbf{x}(0) + t\dot{\mathbf{x}}(0) + \frac{1}{M} \int_0^t \int_0^\tau \mathbf{F}(\alpha) d\alpha d\tau \end{aligned}$$

其中， $\mathbf{x}(0)$  为初始位置。利用这些方程，如果已知一个物体的初始位置和初速度，以及施加在这个物体上的三个方向的以时间函数表示的作用力，那么可以决定这个物体在任意时刻的加速度、速度和位置。

以上作用于方向的运动方程可用力矩，即旋转形式的作用力来表示。力矩是一个时间函数的三元矢量，表示作用在一个物体上的净旋转力(net rotational force)。它与角速度相关，类似于式(2.1)，得到

<sup>⊖</sup> 连续时间信号的定义域限于  $\mathbb{R}$  的相关子集，如  $\mathbb{R}_+$  (即非负实数)，或  $[0, 1]$  区间内的所有数。值域可以是一个任意集合，当然表示物理量时，实数更加有用。

$$\mathbf{T}(t) = \frac{d}{dt}(\mathbf{I}(t)\dot{\boldsymbol{\theta}}(t)) \quad (2.2)$$

其中,  $\mathbf{T}$  是 3 个轴上的力矩矢量,  $\mathbf{I}(t)$  是物体的惯性矩张量(moment of inertia tensor)。惯性矩是一个  $3 \times 3$  的矩阵, 取决于物体的几何形状和方向。直观上, 惯性矩作为沿三个轴的方向函数, 表示物体绕任意轴旋转的磁阻(reluctance)。如果物体是球形的, 所有轴上的磁阻都是相同的, 因而可以简化为一个常量  $I$  (或者等价地表示成对角线元素为  $I$  的一个对角矩阵  $\mathbf{I}$ )。方程式在形式上如式(2.1),

$$\mathbf{T}(t) = \mathbf{I}\ddot{\boldsymbol{\theta}}(t) \quad (2.3)$$

为了更加明确地描述三维空间, 式(2.2)可以写成

$$\begin{bmatrix} T_x(t) \\ T_y(t) \\ T_z(t) \end{bmatrix} = \frac{d}{dt} \begin{bmatrix} I_{xx}(t) & I_{xy}(t) & I_{xz}(t) \\ I_{yx}(t) & I_{yy}(t) & I_{yz}(t) \\ I_{zx}(t) & I_{zy}(t) & I_{zz}(t) \end{bmatrix} \begin{bmatrix} \dot{\theta}_x(t) \\ \dot{\theta}_y(t) \\ \dot{\theta}_z(t) \end{bmatrix}$$

其中,  $T_y(t)$  是绕  $y$  轴的净力矩(将导致方位角的变化),  $I_{yx}(t)$  是惯性, 确定绕  $x$  轴的加速度与绕  $y$  轴的力矩的关系。

旋转速度是加速度的积分,

$$\dot{\boldsymbol{\theta}}(t) = \dot{\boldsymbol{\theta}}(0) + \int_0^t \ddot{\boldsymbol{\theta}}(\tau) d\tau,$$

其中,  $\dot{\boldsymbol{\theta}}(0)$  是三个轴上的初始旋转速度。对于球形物体, 利用式(2.3), 上式变成

$$\dot{\boldsymbol{\theta}}(t) = \dot{\boldsymbol{\theta}}(0) + \frac{1}{I} \int_0^t \mathbf{T}(\tau) d\tau$$

方向是旋转速度的积分,

$$\begin{aligned} \boldsymbol{\theta}(t) &= \boldsymbol{\theta}(0) + \int_0^t \dot{\boldsymbol{\theta}}(\tau) d\tau \\ &= \boldsymbol{\theta}(0) + \dot{\boldsymbol{\theta}}(0)t + \frac{1}{I} \int_0^t \int_0^\tau \mathbf{T}(\alpha) d\alpha d\tau \end{aligned}$$

其中,  $\boldsymbol{\theta}(0)$  是初始方向。利用这些方程式, 如果已知初始方向、物体的初始旋转速度以及从三个方向作用于物体的力矩的时间函数, 就可以确定这个物体在任意时刻的旋转加速度、速度以及方向。

大多数情况下, 如前面给出的球形物体, 可以通过减少维数来进行简化处理。一般来说, 这种简化称为模型降阶(model-order reduction)。例如, 如果物体是运行在一个平面上的车辆, 那么可以不考虑物体在  $y$  轴上的移动、俯仰角或横滚角。

**例 2.1:** 考虑一个可以减少维数的简单控制问题。直升机有两个旋翼, 一个在正上方, 提供上升力, 另一个在机尾。如果机尾没有旋翼, 直升机的机身部分将会开始自旋。机尾的旋翼可以中和自旋力。尤其是, 机尾旋翼产生的动力必须与主机翼产生的力矩完全抵消, 否则机身将自旋。这里所考虑的尾翼的作用独立于直升机的所有其他运动。

直升机的高度简化模型如图 2-2 所示。在这个形式中, 假设直升机位置固定于原点, 因而没有必要考虑受制于动态位置的方程式。此外, 假设直升机保持垂直, 因此俯仰角和横滚角可以设为 0 度。值得注意的是, 这些假设并不是不切实际的, 因为可以给直升机定义一个

坐标系。

有了这些假设，惯性矩可以简化为一个标量，表示反作用于方位角变化的力矩。引起方位角变化的力矩是由于主旋翼的摩擦，因而导致直升机沿着同一方向旋转，表现为旋翼的旋转。尾旋翼的任务是反作用于主旋翼产生的力矩，从而防止直升机自旋。

通过一个输入为连续时间信号  $T_y$  的系统对简化直升机进行建模， $T_y$  是绕  $y$  轴的力矩（导致方位角变化），表示主旋翼摩擦产生的力矩和尾旋翼摩擦产生的力矩之间的净差值。系统的输出是绕  $y$  轴的角速度  $\dot{\theta}_y$ 。式(2.2)的降维形式为

$$\ddot{\theta}_y(t) = T_y(t) / I_{yy}$$

对两边同时进行积分，得到一个输入为  $T_y$  的函数的输出  $\dot{\theta}$ ，

$$\dot{\theta}_y(t) = \dot{\theta}_y(0) + \frac{1}{I_{yy}} \int_0^t T_y(\tau) d\tau \quad (2.4)$$

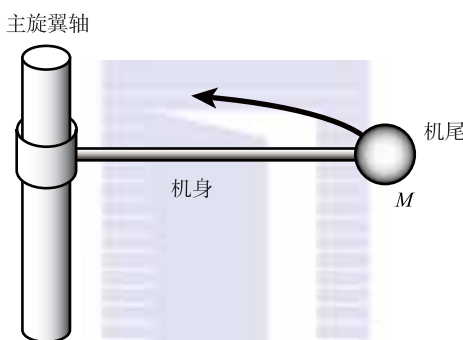
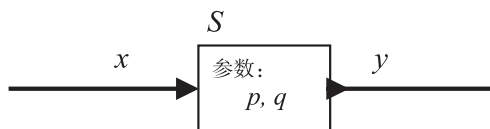


图 2-2 直升机的简化模型

由以上实例可知，如果选择令  $\mathbf{x} : \mathbb{R} \rightarrow \mathbb{R}^3$  表示直升机机尾空间的绝对位置来对直升机进行建模，最终会得到一个更为复杂的模型，从而设计控制系统也会更加困难。

## 2.2 参量模型

前一节以微分方程或积分方程的形式给出了一个物理系统模型，这些方程式建立输入信号（力或力矩）与输出信号（位置、方向、速度或旋转速度）之间的关联。这样的物理系统可以被视为一个大系统里的一个元件。特别是，一个连续时间系统（处理的是连续时间信号）可以利用一个具有输入端口和输出端口的盒子进行建模，如下图所示。



其中，输入信号  $x$  和输出信号  $y$  都是基于以下形式的函数：

$$x : \mathbb{R} \rightarrow \mathbb{R}, y : \mathbb{R} \rightarrow \mathbb{R}$$

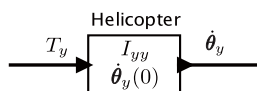
上式中定义域表示时间，值域表示在特定时刻的信号值。如果想要明确地建立一个能够实现并在某个特定时刻开始运行的系统模型，则定义域  $\mathbb{R}$  可由非负实数  $\mathbb{R}_+$  来替代。

系统的模型是一个函数，形如：

$$S : X \rightarrow Y \quad (2.5)$$

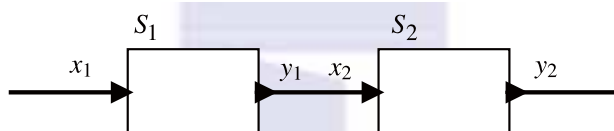
其中,  $X=Y=\mathbb{R}^{\mathbb{R}}$ , 是从实数映射到实数的函数集, 如上式中的  $x$  与  $y$  <sup>⊖</sup>。函数  $S$  可能依赖于系统的参数, 在这种情况下参数可以有选择地显示在盒子中, 也可以有选择地包含在函数标记符中。例如, 在上图中, 如果有参数  $p$  和  $q$ , 则可以把系统函数写成  $S_{p,q}$  或写成  $S(p, q)$ , 两种标记都是式(2.5)的表示形式。如上图的盒子, 输入是函数, 而输出也是函数, 这个盒子称为**参量**(actor)。

例 2.2: 例 2.1 直升机的参量模型可以描述如下:



输入和输出都是连续时间函数。这个参量的参数为初始角速度  $\dot{\theta}_y(0)$  和惯性矩  $I_{yy}$ 。参量函数如式(2.4)定义。

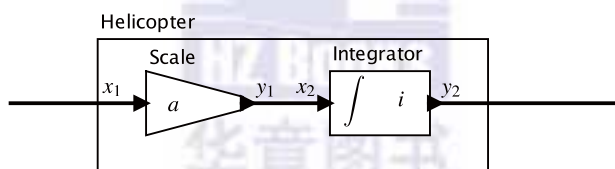
参量模型是可以组合的。特别地, 给出两个参量  $S_1$  和  $S_2$ , 可以形成如下的级联组合 (cascade composition):



图中,  $S_1$  的输出和  $S_2$  的输入之间的连线说明  $y_1 = x_2$ , 或者更加符合学术规范写成:

$$\forall t \in \mathbb{R}, y_1(t) = x_2(t)$$

例 2.3: 直升机的参量模型可以表述为两个参量的级联形式, 如:



左边的参量表示一个定标器(Scale)参量, 由常数  $a$  进行参数化, 定义为:

$$\forall t \in \mathbb{R}, y_1(t) = ax_1(t) \quad (2.6)$$

更严密地, 可以写成  $y_1 = ax_1$ , 显而易见地, 标量  $a$  和函数  $x_1$  的乘积如式(2.6)所示。右边的参量表示一个积分器(Integrator), 由初始值  $i$  进行参数化, 定义为:

$$\forall t \in \mathbb{R}, y_2(t) = i + \int_0^t x_2(\tau) d\tau$$

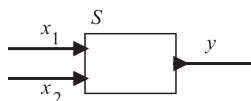
如果给定参数值  $a=1/I_{yy}$  和  $i=\dot{\theta}_y(0)$ , 系统表示的是式(2.4), 其中输入  $x_1=T_y$  是力矩, 输出  $y_2=\dot{\theta}_y$  是角速度。

上图中, 给出了自定义的**图标**, 即用盒子表示参量。这些特定的参量(如定标器和积分器)作为非常有效的构建模块, 用于构建物理动态模型, 因此采用容易记忆的符号也是十分有用的。

参量可以有多输入信号或多输出信号。这些都可以同样表示, 如下面的例子, 其中有两

⊖ 正如附录 A 所解释的, 标记符  $\mathbb{R}^{\mathbb{R}}$  (也可以写成  $(\mathbb{R} \rightarrow \mathbb{R})$ ) 表示定义域和值域都是实数集  $\mathbb{R}$  的所有函数的集合。

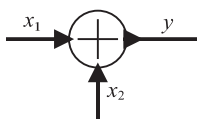
个输入信号和一个输出信号：



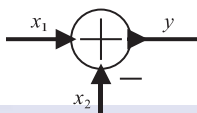
这种形式的有效模块是信号加法器(adder)，定义为：

$$\forall t \in \mathbb{R}, y(t) = x_1(t) + x_2(t)$$

常常用自定义图标表示为：



有时有一个输入是减法而不是加法，这种情形下在图标的该输入端标识为负号，如：



这个参量表示函数  $S : (\mathbb{R} \rightarrow \mathbb{R})^2 \rightarrow (\mathbb{R} \rightarrow \mathbb{R})$ ，由下式给出：

$$\forall t \in \mathbb{R}, \forall x_1, x_2 \in (\mathbb{R} \rightarrow \mathbb{R}), (S(x_1, x_2))(t) = y(t) = x_1(t) - x_2(t)$$

特别注意这个标记。 $S(x_1, x_2)$ 是一个属于 $\mathbb{R}^{\mathbb{R}}$ 的函数。因此，它可以作用于 $t \in \mathbb{R}$ 。

除非讨论需要，本章的其余部分对系统及其参量模型将不进行区分。假设参量模型具有系统的所有特性。不可否认这是一个大胆的假设。通常情况下，参量模型的特性只是实际系统的近似描述。

## 2.3 系统的特性

本节将考虑参量及其所组成的系统可能包含的特性，其中包括因果关系、无记忆性、线性、时不变性以及稳定性。

### 2.3.1 因果关系系统

直观上，如果一个系统的输出只依赖于当前和过去的输入，那么这个系统是因果关系(causal)系统。然而要使这个概念非常明确，还需要一点小技巧。可以这样想，首先给出一个符号表示“当前和过去的输入”。考虑一个连续的时间信号  $x : \mathbb{R} \rightarrow A$ ， $A$  是一个集合。令  $x|_{t \leq \tau}$  表示一个称为时间约束(restriction in time)的函数，只定义于时间  $t \leq \tau$ ，并且满足  $x|_{t \leq \tau}(t) = x(t)$ 。因此，如果  $x$  是系统的一个输入，那么  $x|_{t \leq \tau}$  表示在  $\tau$  时刻的“当前和过去的输入”。

考虑一个连续时间系统  $S : X \rightarrow Y$ ，其中对于集合  $A$  和  $B$ ， $X = A^{\mathbb{R}}$  和  $Y = B^{\mathbb{R}}$ 。如果对于所有的  $x_1, x_2 \in X$  和  $\tau \in \mathbb{R}$ ，满足下式，那么这个系统是因果关系系统。

$$x_1|_{t \leq \tau} = x_2|_{t \leq \tau} \Rightarrow S(x_1)|_{t \leq \tau} = S(x_2)|_{t \leq \tau}$$

也就是说，如果对于截止到时刻  $\tau$  (包括  $\tau$  在内)都相等的两个可能输入  $x_1$  和  $x_2$ ，截止于时刻  $\tau$  (包括  $\tau$  在内)的输出也相等，那么这个系统是因果关系系统。到目前为止我们所考虑的系统都是因果关系系统。

如果对于所有的  $x_1, x_2 \in X$  和  $\tau \in \mathbb{R}$ ，满足下式，那么这个系统是严格因果关系的(strictly causal)。

$$x_1|_{t \leq \tau} = x_2|_{t \leq \tau} \Rightarrow S(x_1)|_{t \leq \tau} = S(x_2)|_{t \leq \tau}$$

也就是说, 如果对于截止到时刻  $\tau$  (不包括  $\tau$  在内) 都相等的两个可能输入  $x_1$  和  $x_2$ , 截止到时刻  $\tau$  (包括  $\tau$  在内) 的输出也相等, 那么这个系统是严格因果关系系统。严格因果关系系统中, 在  $\tau$  时刻的输出并不依赖于它在  $t$  时刻的输入。它只取决于过去的输入。当然, 一个严格因果关系系统也是一个因果关系系统。比如, 积分器参量是一个严格的因果关系系统。加法器则不是一个严格的因果关系系统, 但它是一个因果关系系统。严格因果关系参量对于构建反馈系统是十分有益的。

### 2.3.2 无记忆系统

直观上, 如果一个系统的输出不仅取决于它当前的输入, 还取决于它过去的输入 (对于一个非因果系统, 也许输出还取决于其接下来的输入), 那么这个系统具有记忆性。考虑一个连续时间系统  $S: X \rightarrow Y$ , 其中对于集合  $A$  和  $B$ ,  $X = A^{\mathbb{R}}$  和  $Y = B^{\mathbb{R}}$ 。形式上, 如果存在一个函数  $f: A \rightarrow B$  并且对于所有的  $x \in X$ , 都满足

$$(S(x))(t) = f(x(t)), \text{ 其中 } t \in \mathbb{R}$$

那么这个系统是无记忆性的 (memoryless)。也就是说,  $t$  时刻的输出  $(S(x))(t)$  仅取决于  $t$  时刻的输入  $x(t)$ 。

前面提到的积分器不是无记忆性的, 但加法器是无记忆性的。练习 2 表明, 如果一个系统是严格因果关系的并且是无记忆性的, 那么对于所有的输入, 它的输出为一个常量。

### 2.3.3 线性和时不变性

线性和时不变性 (LTI) 系统具有很好的数学特性, 很多控制系统理论都依赖于这些特性。这些数学特性成为“信号与系统”课程的主体, 不在本书所要阐述的范围。不过, 我们有时会提及一些特性的简单形式, 因此有必要确定一个系统何时是线性和时不变的。

如果一个系统  $S: X \rightarrow Y$ , 其中  $X$  和  $Y$  是信号集, 满足下述叠加 (superposition) 特性, 则它是线性的。

$$\forall x_1, x_2 \in X \text{ 和 } \forall a, b \in \mathbb{R}, S(ax_1 + bx_2) = aS(x_1) + bS(x_2)$$

显而易见, 例 2.1 所示的直升机系统当且仅当初始角速度  $\dot{\theta}_y(0) = 0$  时是线性的 (见练习 3)。

更一般地, 显而易见, 例 2.3 定义积分器当且仅当初始值  $i = 0$  时是线性的, 而定标器参量总是线性的, 任意两个线性参量的串联是线性的。自然地, 可以将线性的定义扩展到多输入或多输出的参量, 由此可以确定加法器也是线性的。

为定义时不变性, 首先定义一个称为时延 (delay) 的专门连续时间参量。令  $D_\tau: X \rightarrow Y$ , 其中  $X$  和  $Y$  是连续时间信号的集合, 定义为:

$$\forall x \in X \text{ 和 } \forall t \in \mathbb{R}, (D_\tau(x))(t) = x(t - \tau) \quad (2.7)$$

其中,  $\tau$  是时延参量的参数。如果一个系统  $S: X \rightarrow Y$  满足下式, 则它是时不变的。

$$\forall x \in X \text{ 和 } \forall \tau \in \mathbb{R}, S(D_\tau(x)) = D_\tau(S(x))$$

例 2.1 和式 (2.4) 所定义的直升机系统不是时不变的。不过, 它的简化形式是时不变的:

$$\dot{\theta}_y(t) = \frac{1}{I_{yy}} \int_{-\infty}^t T_y(\tau) d\tau$$

该形式不允许有初始的角旋转。

线性的时不变系统 (Linear Time-Invariant system, LTI) 是一个同时满足线性和时不变

性的系统。物理动态建模的一个主要目标是尽可能选择一个 LTI 模型。如果利用 LTI 模型可以得到一个合理的近似结果，那么很值得进行这样的近似。当然，确定这个近似值是否合理或者是否找到一个合理的近似模型也不是一件容易的事。通常，构造的模型比所需要的更为复杂(见练习 4)。

### 2.3.4 稳定性

如果对于所有的有界输入信号，输出信号也是有界的，那么这样的系统被看做是“有界输入有界输出稳定的”(BIBO stable)或简称“稳定的”。

考虑一个输入为  $w$ 、输出为  $v$  的连续时间系统。如果存在一个实数  $A < \infty$ ，对于所有的  $t \in \mathbb{R}$ ，都有  $|w(t)| \leq A$ ，则输入是有界的。同样，如果存在一个实数  $B < \infty$ ，对于所有的  $t \in \mathbb{R}$ ，都有  $|v(t)| \leq B$ ，则输出是有界的。如果对于由实数  $A$  界定的任意输入，输出都受实数  $B$  界定，那么这个系统是稳定的。

**例 2.4：**现在很容易理解例 2.1 提到的直升机系统是不稳定的。令输入为  $T_y = u$ ，其中  $u$  是单位阶跃信号，可表示为：

$$\forall t \in \mathbb{R}, u(t) = \begin{cases} 0, & t < 0 \\ 1, & t \geq 0 \end{cases} \quad (2.8)$$

也就是说，在零时刻之前，没有力矩输入系统，从零时刻开始，加入一个单位强度的力矩。显然，这个输入是有界的，不会超过一个单位强度。然而，它的输出可以无界限地增长。

实际上，直升机利用一个反馈系统来确定需要多少力矩作用于尾旋翼，以保持直升机的机体平衡。接下来将研究如何实现这种反馈。

## 2.4 反馈控制

一个带有反馈的系统有定向循环，其中一个参量的输出被反馈回去影响同一参量的输入。如图 2-3 所示。大多数控制系统都使用反馈。需要计算误差(在图中用  $e$  表示)，即期望行为(在图中用  $\psi$  表示)与实际行为(在图中用  $\dot{\theta}_y$  表示)之间的差值，并用误差量来校正行为。误差量就是反馈，相应的校正信号(图中的  $T_y$ )进行补偿，从而减少下一步的误差。值得注意的是，校正信号通常只能影响下一步的误差，因此一个反馈系统通常在每个定向循环中至少包括一个严格因果关系参量(图中的直升机)。

反馈控制是一个复杂的课题，占据很多教材和完整课程的内容。这里仅略谈一下这个课题，只限于软件与物理系统之间的相互作用。反馈控制系统常常利用嵌入式软件来实现，而整个物理动态是一个软件与物理动态的组合物。Lee and Varaiya(2003)的第 12~14 章讲述得非常详细。

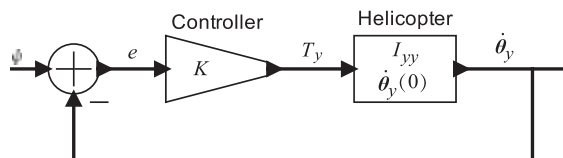


图 2-3 稳定直升机的比例控制系统

**例 2.5：**前面提到，例 2.1 中的直升机模型是不稳定的。可以通过一个简单的反馈控制

系统使其变稳定,如图 2-3 所示。该系统的输入  $\psi$  是一个具有期望角速度的连续时间系统。误差信号  $e$  表示实际角速度与期望角速度之间的差值。图中,控制器通过常量  $K$  简单调节误差信号,并提供给直升机一个输入控制信号。用式(2.4)表示为

$$\dot{\theta}_y(t) = \dot{\theta}_y(0) + \frac{1}{I_{yy}} \int_0^t T_y(\tau) d\tau \quad (2.9)$$

$$= \dot{\theta}_y(0) + \frac{K}{I_{yy}} \int_0^t (\psi(\tau) - \dot{\theta}_y(\tau)) d\tau \quad (2.10)$$

其中用到如下依据(从图中):

$$e(t) = \psi(t) - \dot{\theta}_y(t) \text{ 和 } T_y(t) = Ke(t)$$

由于式(2.10)两边都有  $\dot{\theta}_y(t)$ , 因而不好求解。最简单的求解方法是利用拉普拉斯变换(详见 Lee and Varaiya(2003)的第 14 章)。然而,这里的目的是可以利用微积分中更加强有力的求解方法。为了使这个方法尽可能简单,假设对于所有的  $t$ ,  $\psi(t)=0$ ;也就是说,想要简单地控制直升机以防止其自旋。其期望角速度是零。这样,式(2.10)简化为:

$$\dot{\theta}_y(t) = \dot{\theta}_y(0) - \frac{K}{I_{yy}} \int_0^t \dot{\theta}_y(\tau) d\tau \quad (2.11)$$

利用微积分的理论,对于  $t \geq 0$ ,

$$\int_0^t a e^{a\tau} d\tau = e^{at} u(t) - 1$$

其中  $u$  由式(2.8)给出,可以推导出式(2.11)的解为

$$\dot{\theta}_y(t) = \dot{\theta}_y(0) e^{-Kt/I_{yy}} u(t) \quad (2.12)$$

(值得注意的是,尽管很容易证明这个解是正确的,但推导出这个解并不容易。为此,拉普拉斯变换是一个更好的解决方法。)

从式(2.12)得知,随着  $t$  不断增大,同时系数  $K$  为正值,角速度趋向于期望角速度(零值)。对于一个更大的  $K$  值,这种趋向会更快。而对于一个负值  $K$ ,系统是不稳定的,角速度会无界限增长。

前面的实例显示的是一个比例控制反馈回路,之所以这样命名是由于控制信号与误差是成比例的。假设有一个零值期望信号。同样,很容易假设直升机初始为静止状态(它的角速度为零),然后确定一个特定的非零期望信号的行为,如下面所举的例子所示。

**例 2.6:** 假设直升机最初为静止状态,即

$$\dot{\theta}(0) = 0$$

对于某个常量  $a$ , 期望信号为

$$\psi(t) = au(t)$$

也就是说,想要控制直升机以一个恒定的速率旋转。

利用式(2.4),可写成:

$$\dot{\theta}_y(t) = \frac{1}{I_{yy}} \int_0^t T_y(\tau) d\tau$$

$$\begin{aligned}
&= \frac{K}{I_{yy}} \int_0^t (\psi(\tau) - \dot{\theta}_y(\tau)) d\tau \\
&= \frac{K}{I_{yy}} \int_0^t a d\tau - \frac{K}{I_{yy}} \int_0^t \dot{\theta}_y(\tau) d\tau \\
&= \frac{Kat}{I_{yy}} - \frac{K}{I_{yy}} \int_0^t \dot{\theta}_y(\tau) d\tau
\end{aligned}$$

利用同样的推断和验证方法，可以得到解为

$$\dot{\theta}_y(t) = au(t)(1 - e^{-Kt/I_{yy}}) \quad (2.13)$$

同样，随着时间  $t$  变大，且  $K$  为正值，角速度趋向于期望角速度。 $K$  值变大，这种趋向也变得更快速。如果  $K$  为负值，系统变得不稳定，且角速度会无界限增长。

值得注意的是，上述解的第一项正是期望角速度。第二项是跟踪误差(tracking error)，对于这个例子而言，跟踪误差逐渐趋向于零。

上述实例有些不切实际，因为直升机的净力矩是无法独立控制的。特别是，净力矩  $T_y$  是力矩  $T_l$  和力矩  $T_r$  的总和，其中  $T_l$  由主旋翼的摩擦产生， $T_r$  由尾旋翼的摩擦产生。

$$\forall t \in \mathbb{R}, T_y(t) = T_l(t) + T_r(t)$$

$T_l$  可以由保持或达到某一期望高度所需的旋转速度来决定，这一点基本上独立于直升机的旋转。因此，实际上需要设计的只是一个控制  $T_r$  并对任意的  $T_l$  (更加精确地说，是操作参数中的任意  $T_l$ ) 而言可以保证直升机稳定的控制系统。下面的例子研究如何改变控制系统的性能。

**例 2.7:** 如图 2-4a 所示，对直升机模型做些改变，使它有两个输入  $T_l$  和  $T_r$ ，分别对应主旋翼和尾旋翼的力矩。这样一来，反馈控制系统只控制力矩  $T_r$ ，而  $T_l$  被视做一个外部(不受控制的)输入信号。这个控制系统将如何运行呢？

同样，这个问题的整个处理超出了本书的范围，这里只研究一个具体的例子。假设主旋翼产生的力矩对于某个常数  $b$  满足

$$T_l = bu(t)$$

也就是说，在零时刻，主旋翼以一个恒定速度开始旋转，并保持那个速度。进一步假设直升机初始状态是静止状态。可以利用例 2.6 的结论来得到系统的行为。

首先，将这些模型转换为一个等价的模型，如图 2-4b 所示。这种转换仅依赖于代数规则，即对于任意的实数  $a_1$ 、 $a_2$ 、 $K$ ，满足

$$Ka_1 + a_2 = K(a_1 + a_2/K)$$

如进一步转换模型为一个等价模型，图 2-4c 所示，其中用到加法交换律。由图 2-4c 可知，封闭在盒子中的模型的一部分与例 2.6 中分析的控制系统十分相似，如图 2-3 所示。因此，例 2.6 的分析方法同样适用于此。假设期望的角旋转为

$$\phi(t) = 0$$

原控制系统的输入变为

$$x(t) = \phi(t) + T_l(t)/K = (b/K)u(t)$$

从式(2.13)，得到解为

$$\dot{\theta}_y(t) = (b/K)u(t)(1 - e^{-Kt/I_{yy}}) \quad (2.14)$$

期望的角旋转是零，但是控制系统逐渐趋向于一个非零角旋转  $b/K$ 。跟踪误差可通过使

控制系统反馈增益  $K$  值变大而控制到任意大小，但是设计这样一个控制器，跟踪误差无法为零。练习 6 设计的另一种控制器，跟踪误差逐渐趋近于零。

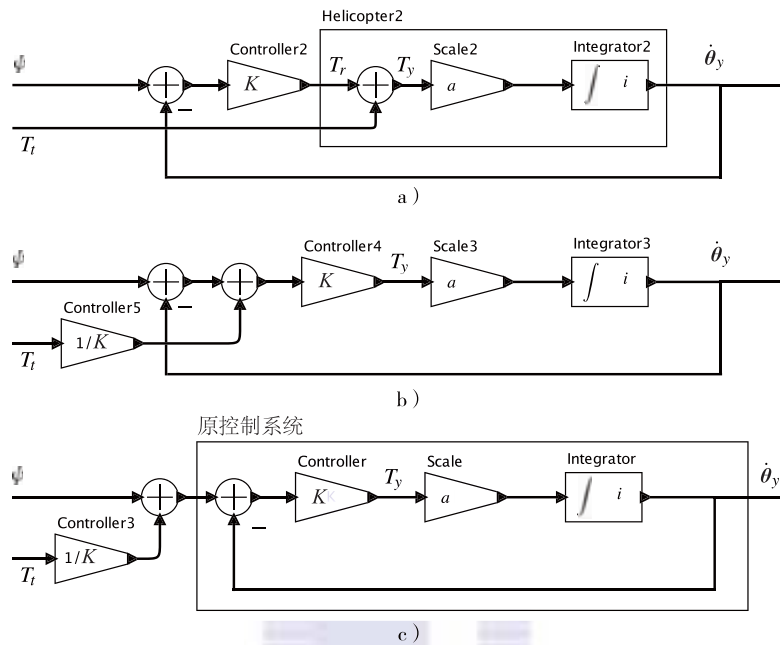


图 2-4 a) 主旋翼和尾旋翼各有一个控制力矩的直升机模型；b) 等价模型的变换(假设  $K>0$ )；c) 用于理解控制器行为的进一步的等价模型变换

2.5 小结

本章讲述了两截然不同的建模方法来描述物理动态。第一种是常微分方程，是工程师不可或缺的工具；第二种是参量模型，是由软件建模与仿真工具驱动的较新方法。这两种方法密切相关。本章主要强调了这些模型之间的相互关系，以及这些模型与待建模的系统之间的关系。然而，这些关系相当深奥，我们几乎很少涉及。本书的目的是让读者理解一个系统可以利用多个模型进行建模，而且这些模型与所建模的系统是有区别的。模型的忠实度(fidelity，即多大程度上与所建模的系统近似)是关系到一个工程计划成败的关键因素。

练习

1. 如图 2-5 所示，音叉是由一个金属手指(称为叉)构成的，当用一个锤敲击时，它会产生位移。产生位移后就会颤动。如果音叉没有摩擦，它会一直颤动不停。将在零时刻敲击后音叉的位移标记为一个函数  $y: \mathbb{R}_+ \rightarrow \text{Reals}$ 。假设由锤敲击产生的初始位移是一个单位，那么运用已学的物理知识，可以确定对于所有  $t \in \text{Reals}_+$ ，位移满足微分方程

$$\ddot{y}(t) = -\omega_0^2 y(t)$$

其中  $\omega_0^2$  是个常数，取决于叉的质量和刚度，而  $\ddot{y}(t)$  是  $y$  相对于时间的二阶导数。很容易证明  $y$  由下式给出

$$\forall t \in \text{Reals}_+, y(t) = \cos(\omega_0 t)$$

上式就是一个微分方程的解(只需求得它的二阶导数)。因此，音叉的位移是正弦曲线。如果选择的音叉材质使  $\omega_0 = 2\pi \times 440$  弧度/秒，那么音叉会产生 A-440 的音阶。

(a)  $y(t) = \cos(\omega_0 t)$  是否是唯一解？如果不是，给出其他的解。

- (b) 假设解为  $y(t) = \cos(\omega_0 t)$ ，求它的初始位移。
- (c) 建立一个音叉模型，要求将  $y$  作为输出，使用通用的参量如积分器、加法器、定标器，或类似的简单参量。把初始位移看做一个参数。在图中进行详细标记。
2. 说明如果一个系统  $S: \mathbb{R}^A \rightarrow \mathbb{R}^B$  是严格因果关系的和无记忆的，那么它的输出是恒定的。所谓恒定是指  $t$  时刻的输出  $(S(x))(t)$  不依赖于  $t$  值。
3. 本题研究的是线性特性。
- (a) 说明当且仅当初始角速度  $\dot{\theta}_y(0) = (0)$  时，例 2.1 定义的直升机模型是线性的。
- (b) 说明任意两个线性参量的串联是线性的。
- (c) 对线性的定义进行扩展，使它适用于一个有两个输入信号和一个输出信号的参量。说明这样的加法器参量是线性的。
4. 思考例 2.1 中的直升机，不过其输入和输出的定义稍有不同。假设输入与例子相同，为  $T_y: \mathbb{R} \rightarrow \mathbb{R}$ ，然而输出为相对于主旋翼轴的机尾位置。这样的模型是否是 LTI 的？它是否是 BIBO 稳定的？
5. 考虑一个旋转机器人，可以绕一个固定轴控制其角速度。
- (a) 建立一个系统模型，其输入为角速度  $\dot{\theta}$ ，输出为角度  $\theta$ 。以时间函数的形式建立输入与输出的关系方程来表达该模型。
- (b) 这个模型是否是 BIBO 稳定的？
- (c) 设计一个比例控制器使机器人得到一个期望角度。也就是说，假设初始角度  $\theta(0) = 0$ ，令期望角度为  $\phi(t) = au(t)$ 。找出实际角度的时间函数和比例控制器的反馈增益  $K$ 。试给出  $t=0$  时刻的输出。随着  $t$  的增大，它趋向于什么？
6. (a) 使用你最擅长的连续时间建模软件（如 LabVIEW、Simulink 或 Ptolemy II），构建一个如图 2-4 所示的直升机控制系统模型。选择一些合理的参数，将实际角速度绘制成时间函数图，假设期望角速度为零，即  $\phi(t) = 0$ ，且主旋翼力矩为非零，即  $T_r(t) = bu(t)$ 。绘制不同  $K$  值时的曲线，并讨论在不同  $K$  值时系统的行为是如何变化的。
- (b) 将(a)中的模型(由  $K$  调节大小的简单参量)换成如图 2-6 所示的控制器。这个控制器称为**比例积分控制器**(proportional-integrator (PI) controller)。它有两个参数  $K_1$  和  $K_2$ 。利用这两个参数进行实验，绘制与(a)相同输入时的行为曲线，并相对于(a)讨论这个控制器的行为。

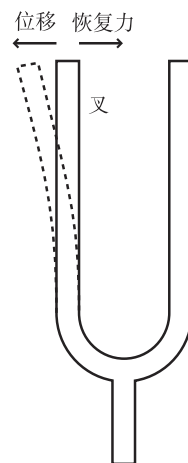


图 2-5 音叉

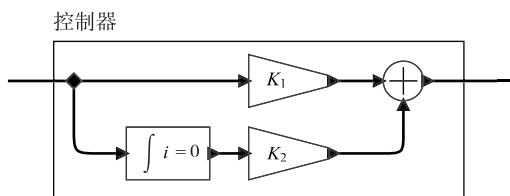


图 2-6 直升机的 PI 控制器

## 离散动态

嵌入式系统模型包括**离散组件**和**连续组件**。简单来说，连续组件是平缓(smoothly)变化的，而离散组件是突变的(abruptly)。上一章中已经研究了连续组件，表明系统的物理动态(physical dynamics)通常可以利用常微分方程或积分方程，以及映射这些方程的等价参量模型进行建模。另一方面，离散组件并不方便利用常微分方程进行建模。本章将研究如何利用状态机对离散动态进行建模。下一章将展示如何把这些状态机与连续动态模型组合在一起得到混合系统模型。

### 3.1 离散系统

**离散系统**(discrete system)是按照一系列离散的步骤进行操作，所谓具有**离散动态**(discrete dynamics)。有些系统自身就是离散的。

**例 3.1**：考虑这样的一个系统，计算停车场里进出的车辆，以便跟踪记录任意时刻停车场里车辆的数目。可以建立一个如图 3-1 所示的模型。先不考虑如何设计用于检测进出车辆的传感器。只简单地假设有一辆车到达时，参量 ArrivalDetector 产生一个事件，车辆离开时，参量 DepartureDetector 也产生一个事件。参量 Counter 从初始值  $i$  开始不断计数。每次当计数有变化，则产生一个输出事件来更新显示。

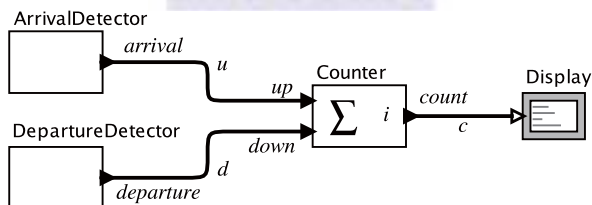


图 3-1 停车场车辆计数系统模型

以上例子中，每次进或出都利用一个**离散事件**来建模。离散事件发生在瞬间，而非随着时间推移。图 3-1 所示的 Counter 参量模拟的是上一章所使用的 Integrator 参量，如图 3-2 所示。如 Counter 参量一样，Integrator 对输入值进行累积，但做法却很不一样。Integrator 的输入为函数  $x: \mathbb{R} \rightarrow \mathbb{R}$  或者  $x: \mathbb{R}_+ \rightarrow \mathbb{R}$ ，是一个连续时间信号。另一方面，发送到计数器的  $up$  输入端的信号  $u$  是下面的函数形式：

$$u: \mathbb{R} \rightarrow \{absent, present\}$$

也就是说，在对于任何时间  $t \in \mathbb{R}$ ，输入  $u(t)$  或者为 *absent*，表示该时刻没有事件发生；或者为 *present*，表示有事件发生。这种形式的信号称为**纯信号**。纯信号不具有任何值，却通过任意给定时间的 *present* 或 *absent* 来提供信息。图 3-1 中的信号  $d$  也是一个纯信号。

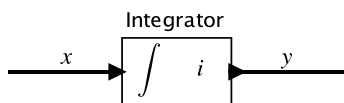


图 3-2 上一章使用的参量 Integrator 图标

假设计数器 Counter 运行如下。当一个事件在 *up* 输入端为 *present* 时，递加计数，并产生新的计数值作为输出。当事件在 *down* 输入端为 *present* 时，递减计数，并产生新的计数值作为输出<sup>⊖</sup>。其他所有时间里(即两个输入都为 *absent* 时)，不产生输出(*count* 没有输出)。因此，图 3-1 中的信号 *c* 可以利用下面形式的函数来建模(符号表示参见附录 A)：

$$c : \mathbb{R} \rightarrow \{\text{absent}\} \cup \mathbb{Z}$$

它不是一个纯信号，但与信号 *u* 和 *d* 一样，它可以是 *absent* 或 *present*。但与信号 *u* 和 *d* 不同的是，当它为 *present* 时，会有一个(整数)值。

进一步假设多数情形下的输入都为 *absent*，或更确切地说，输入是离散的(参见补充资料“进一步探讨：离散信号”)。然后 Counter 对一系列输入事件按顺序进行响应。这与对连续输入进行响应的 Integrator 有很大不同。

Counter 的输入是一对离散信号，在特定时刻有一个事件(处于 *present* 状态)，而在其他时刻没有事件(处于 *absent* 状态)。其输出也是一个离散信号，当输入为 *present* 时有一个自然数的输出值，而其他时刻间则没有<sup>⊖</sup>。显然，当输入为 *absent* 时，Counter 没有必要做任何事情。它仅需要在输入出现时运行。因此，它具有离散动态。

### 进一步探讨：离散信号

离散信号由时间上的一系列即时事件组成。在这里，我们对这个直观的概念进行准确的定义。

考虑具有形式  $e : \mathbb{R} \rightarrow \{\text{absent}\} \cup X$  的一个信号，其中  $X$  是任意一组值的集合。直观上，如果多数时刻没有信号，而有些时刻有信号，可以按顺序计数，那么它是一个离散信号。每次出现信号时，就得到一个离散事件。

能够按顺序对事件进行计数非常重要。例如，如果对于所有有理数  $t$ ， $e$  都为 *present*，那么这个信号不是离散的。它出现的次数无法按顺序计数。直观上，它并不是时间上的即时事件序列(它是时间上的即时事件集，而不是一个序列)。

形式上，令  $T \subseteq \mathbb{R}$  为  $e$  出现的次数集。具体有

$$T = \{t \in \mathbb{R} : e(t) \neq \text{absent}\}$$

如果存在一对一的保序(order preserving)函数  $f : T \rightarrow \mathbb{N}$ ，那么  $e$  是离散的。简单来说，保序是指对于所有的  $t_1, t_2 \in T (t_1 \leq t_2)$ ，都有  $f(t_1) \leq f(t_2)$ 。这样的一对一函数可以确保按时间顺序对事件进行计数。离散信号的一些特性可参见练习 6。

离散系统的动态可以描述为一系列响应步骤，且每个响应都是即时的。离散系统的响应是由离散系统运行的外部条件引发的。在图 3-1 的例子中，出现一个或多个输入事件时，

⊖ 设计带有故障处理器的系统将是个明智的选择，当计数值小于 0 时可以进行恰当的处理。不过在此暂不予考虑。

⊖ 如练习 6 中所示，事实上输入信号是离散的并不一定意味着输出也要是离散的。然而在实际应用中，车辆进出速率上的物理限制确保这些信号是离散的。因此可以安全地假设这些信号是离散的。

Counter 参量响应被引发。也就是说，这个例子中，响应是事件触发的。当计数器的两个输入都不出现时，则没有响应发生。

### 进一步探讨：参量的函数建模

正如 2.2 节中所述，图 3-2 所示的参量 Integrator 可以利用下面的函数建模：

$$I_i : \mathbb{R}_+ \rightarrow \mathbb{R}_+$$

也可以写成：

$$I_i : (\mathbb{R}_+ \rightarrow \mathbb{R}) \rightarrow (\mathbb{R}_+ \rightarrow \mathbb{R})$$

(对于不熟悉的符号请参见附录 A) 图中，

$$y = I_i(x)$$

其中  $i$  为积分器的初始值， $x$  和  $y$  是连续时间信号。例如，设  $i=0$  且对所有的  $t \in \mathbb{R}_+$ ，都有  $x(t)=1$ ，则

$$y(t) = i + \int_0^t x(\tau) d\tau = t$$

同样，图 3-1 所示的参量 Counter 可以通过下面的函数建模：

$$C_i : (\mathbb{R}_+ \rightarrow \{\text{absent}, \text{present}\})^P \rightarrow (\mathbb{R}_+ \rightarrow \{\text{absent}\} \cup \mathbb{Z})$$

其中， $\mathbb{Z}$  为整数且  $P$  是输入端的集合， $P = \{\text{up}, \text{down}\}$ 。前面提到符号  $A^B$  表示从  $B$  到  $A$  的所有函数的集合。因此，函数  $C$  的输入是域为  $P$  的一个函数，对于每个端口  $p \in P$ ，都产生一个函数  $(\mathbb{R}_+ \rightarrow \{\text{absent}, \text{present}\})$ 。反过来，上述函数对于每个时间  $t \in \mathbb{R}_+$ ，都返回  $\text{present}$  或  $\text{absent}$ 。

从一个特定响应可以得到特定时刻  $t$  的输入值，并计算同样时刻  $t$  的输出值。假设一个参量有输入端集合  $P = \{p_1, \dots, p_N\}$ ，其中  $p_i$  为第  $i$  个输入端口。进一步假设对于每个输入端口  $p \in P$ ，集合  $V_p$  表示有输入时端口  $p$  接收的值。 $V_p$  称为端口  $p$  的类型(type)。在响应中，将每个  $p \in P$  看做一个属于  $p \in V_p \cup \{\text{absent}\}$  值的变量。对于每个变量  $p \in P$  而言，输入  $P$  取  $V_p$  中的值或者判断  $p$  为  $\text{absent}$ 。

如果端口  $p$  接收到一个纯信号，那么  $V_p = \{\text{present}\}$ ，为单元素集合(只有一个元素的集合)。信号不缺损时的唯一可能值为  $\text{present}$ 。因此，在响应中，变量  $p$  取集合  $\{\text{present}, \text{absent}\}$  中的一个值。

**例 3.2：**对于停车场计数器，输入端口的集合为  $P = \{\text{up}, \text{down}\}$ 。由于两端口接收的都是纯信号，所以类型为  $V_{\text{up}} = V_{\text{down}} = \{\text{present}\}$ 。如果有一辆车在  $t$  时刻到达而没有车离开，那么响应时  $\text{up} = \text{present}$  而  $\text{down} = \text{absent}$ 。如果有一辆车到达并同时有一辆车离开，那么  $\text{up} = \text{down} = \text{present}$ 。如果都不为真，则二者都为  $\text{absent}$ 。

上述特性同样适用于输出。假设一个离散系统有输出端口  $Q = \{q_1, \dots, q_M\}$ ，且类型为  $V_{q_1}, \dots, V_{q_M}$ 。在每一个响应里，系统对于每一个  $q \in Q$  分配一个值  $q \in V_q \cup \{\text{absent}\}$ ，产生一个输出值。本章将假设时刻  $t$  不发生响应，则输出为  $\text{absent}$ 。因此，离散系统的输出是离散信号。第 4 章将介绍输出不局限于离散的系统(见 3.4 节中的补充资料“Moore 机和 Mealy 机”)。

例 3.3: 图 3-1 所示的参量 Counter 有一个名为 *count* 的输出端口, 所以  $Q = \{count\}$ 。类型为  $V_{count} = \mathbb{Z}$ 。在响应中, *count* 为停车场里的车辆数。

## 3.2 状态的概念

直观上, 一个系统的状态是指它在特定时间点的状况。一般而言, 状态会影响系统对输入做出怎样的响应。形式上, 状态可以定义为对当前或下一个输入产生响应的先前输入的编码。状态是对过去的总结。

考虑图 3-2 所示的参量 Integrator。这个参量所处的状态恰好与任意  $t$  时刻的输出都具有相同的值。它在时刻  $t$  的状态是输入信号积分到时间  $t$  时的值。为了知道子系统在时刻  $t$  或  $t$  之后如何响应, 需要知道时刻  $t$  时的值。我们并不需要知道过去的输入。对于下一步的影响完全可以通过  $t$  时刻的当前值来得到。图 3-2 所示的图标包含  $i$ , 是一个用于某个开始时间的初始状态值。

积分器 Integrator 在一个连续的时间内运行。它累加连续时间的输入信号, 在初始状态的基础上每次生成通过输入所给出的曲线下的累积面积作为输出。任何给定时间的状态是初始状态基础上的累积面积。上一节中介绍的参量 Counter 也有状态, 是过去输入值的累加, 但它是离散运行的。

积分器 Integrator 在时刻  $t$  的状态  $y(t)$  是一个实数。因此, 可以将 Integrator 的状态空间看做  $States = \mathbb{R}$ 。对于图 3-1 所示的 Counter, 时刻  $t$  的状态  $s(t)$  是一个整数, 所以  $States \subset \mathbb{Z}$ 。对于实际的停车场, 拥有有限且非负的  $M$  个空位。因此, 参量 Counter 的状态空间可以表示为:

$$States = \{0, 1, 2, \dots, M\}$$

(这是假设停车场内有多少个空位停多少辆车)。Integrator 的状态空间是无限的(事实上是不可数的)。停车场计数器的状态空间是有限的。有限状态空间的离散模型称为有限状态机(FSM)。对于这些模型有很多有效的分析方法, 接下来讨论。

## 3.3 有限状态机

状态机是一个具有离散动态的系统模型, 每个响应都是基于当前状态的从输入值到输出值的映射。有限状态机(Finite-State Machine, FSM)是一个可能状态集  $States$  为有限的状态机。

如果状态数目非常小, 则 FSM 可以简单地利用如图 3-3 所示的图形符号来表示。图中, 每个状态用一个气泡来表示, 所以对于此图, 状态集为:

$$States = \{State1, State2, State3\}$$

在每个响应的开始, 都有一个初始状态 State1, 图中利用一个悬空箭头来表示。

### 3.3.1 转移

状态间的转移(transition)决定状态机的离散动态以及输入值到输出值的映射。转移用一个弯曲的箭头表示, 从一个状态连到另一个状态, 如图 3-3 所示。转移可以开始和结束于同一个状态, 如图中的 State3。这种情形下的转移称为自转移。

图 3-3 中, 从 State1 到 State2 的转移标记为“判定式/动作”。其中判定式(guard)决定响应时是否有转移发生。动作(action)表明每个响应得到怎样的输出。

判定式是一个谓词(predicate, 一个布尔表达式), 有转移发生时为真, 从转移开始到结束时改变状态。判定式为真时, 有转移发生(enabled)。动作是对输出端口赋一个值(或 *absent*)。有转移时没有提及的输出端口为 *absent*。如果没有任何动作, 则所有输出都为 *absent*。

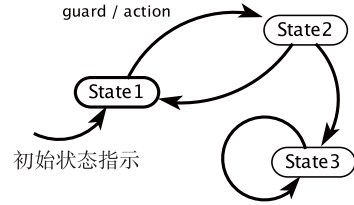


图 3-3 有限状态机的图形符号

例 3.4: 图 3-4 所示的 FSM 模型表示停车场计数器。输入和输出都利用符号 *name : type* 来显示。状态的集合为:  $States = \{0, 1, 2, \dots, M\}$ 。从状态 0 到 1 的转移有判定式, 写成  $up \wedge \neg down$ 。当 *up* 为 *present* 而 *down* 为 *absent* 时, 谓词的值为真。如果响应时当前状态为 0 且判定式的值为真, 则有转移发生, 且下一个状态为 1。此外, 该动作表明输出应该为 1。由于输出端口 *count* 是唯一的, 不会产生歧义, 所以没有明确命名。

从 0 转移到 1 时, 如果判定式只表示为 *up*, 则当 *down* 也为 *present* 时谓词为真, 这样一来, 如果有一辆车到达同时有一辆车离开, 会发生车辆计数错误。

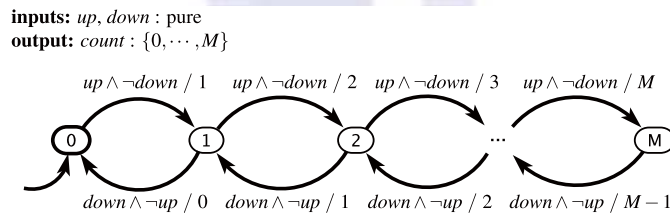


图 3-4 停车场计数器的 FSM 模型

如果离散系统的  $p_1$  和  $p_2$  都是纯输入, 则下面的几个例子为有效的判定式:

|                       |                                                            |
|-----------------------|------------------------------------------------------------|
| $true$                | 总有转移发生。                                                    |
| $p_1$                 | 当 $p_1$ 为 <i>present</i> 时, 有转移发生。                         |
| $\neg p_1$            | 当 $p_1$ 为 <i>absent</i> 时, 有转移发生。                          |
| $p_1 \wedge p_2$      | 当 $p_1$ 和 $p_2$ 都为 <i>present</i> 时, 有转移发生。                |
| $p_1 \vee p_2$        | 当 $p_1$ 和 $p_2$ 有一个为 <i>present</i> 时, 有转移发生。              |
| $p_1 \wedge \neg p_2$ | 当 $p_1$ 为 <i>present</i> 且 $p_2$ 为 <i>absent</i> 时, 有转移发生。 |

这些是标准逻辑运算符, *present* 和 *absent* 分别与 *true* 和 *false* 同义。符号  $\neg$  表示逻辑非。运算符  $\wedge$  表示逻辑与(逻辑 AND), 而  $\vee$  是逻辑或(逻辑 OR)。

假设这个离散系统有第三个输入端口  $p_3$ , 其类型为  $V_{p_3} = \mathbb{N}$ 。下面是几个有效的判定式:

|                      |                                                       |
|----------------------|-------------------------------------------------------|
| $p_3$                | 当 $p_3$ 为 <i>present</i> (不为 <i>absent</i> )时, 有转移发生。 |
| $p_3 = 1$            | 当 $p_3$ 为 <i>present</i> 且值为 1 时, 有转移发生。              |
| $p_3 = 1 \wedge p_1$ | 当 $p_3$ 为 1 且 $p_1$ 为 <i>present</i> 时, 有转移发生。        |
| $p_3 > 5$            | 当 $p_3$ 为 <i>present</i> 且值大于 5 时, 有转移发生。             |

例 3.5: 全球的能源主要用于供热、通风和空调(HVAC)系统。温度变化和温度控制系统的精确模型可以明显节约能源。这种模型源于一个适度的恒温器, 用于将温度调节到一个

设定值(setpoint)或目标温度。单词“恒温器”(thermostat)来自几个希腊单词的组合“热”(hot)和“固守”(to make stand)。

考虑一个恒温器的 FSM 模型, 其状态为  $States = \{\text{heating}, \text{cooling}\}$ , 如图 3-5 所示。假设设定值为 20 摄氏度。开启加热器, 恒温器允许温度高于设定值到 22 度。关闭加热器, 它允许温度低于设定值到 18 度。这种策略称为滞后(hysteresis)(参见“进一步探讨: 滞后”)。这可在温度接近设定值时, 避免因加热器快速开启和关闭而带来的抖动。

加热器有一个  $\mathbb{R}$  类型的单输入  $temperature$  和两个纯输出信号  $heatOn$  和  $heatOff$ 。仅当加热器的状态需要变化时, 这些输出为  $present$ (也就是说, 加热器处于开启状态时需要关闭, 而处于关闭状态时则需要开启)。

与停车场计数器相似, 图 3-5 所示的 FSM 是事件触发的, 每当提供输入  $temperature$  时产生响应。另外, 它也是时间触发的, 也就是说, 它以固定的时间间隔响应。这两种情形下 FSM 的定义都没有改变。恒温器与外部条件有关, 应该产生响应时 FSM 运行。

转移时, 动作(斜杠后面的部分)指定输出端口的结果值。如果  $q_1$  和  $q_2$  为纯输出且  $q_3$  为  $\mathbb{N}$  类型, 则以下几个例子为有效动作:

|                 |                                                                   |
|-----------------|-------------------------------------------------------------------|
| $q_1$           | $q_1$ 为 $present$ 而 $q_2$ 和 $q_3$ 为 $absent$ 。                    |
| $q_1, q_2$      | $q_1$ 和 $q_2$ 都为 $present$ 而 $q_3$ 为 $absent$ 。                   |
| $q_3 := 1$      | $q_1$ 和 $q_2$ 都为 $absent$ 而 $q_3$ 为 $present$ 且值为 1。              |
| $q_3 := 1, q_1$ | $q_1$ 为 $present$ , $q_2$ 为 $absent$ , 而 $q_3$ 为 $present$ 且值为 1。 |
|                 | (全无) $q_1, q_2$ 和 $q_3$ 都为 $absent$ 。                             |

转移时没有提到的任何输出端口都隐含为  $absent$ 。给输出端口赋值时, 可以利用符号  $name := value$ , 而不是  $name = value$ , 来区别谓词。如图 3-1 所示, 如果只有一个输出, 则赋值不需要给出端口名。

### 3.3.2 发生响应时

状态机的定义里没有限制何时响应, 而是由外部条件决定何时发生响应。第 5 章和第 6 章将讲述不同的机制, 并给出诸多术语的确切定义, 如事件触发和时间触发。然而本节只关注状态机响应时会做什么。

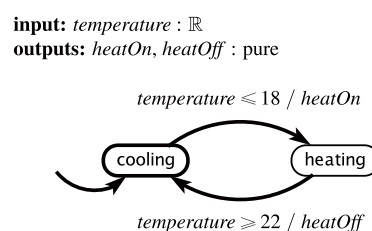


图 3-5 具有滞后功能的恒温器模型

#### 进一步探讨: 滞后

例 3.5 所示的恒温器是状态依赖行为的特殊形式, 称为滞后。滞后是用来防止抖动的。滞后系统是有记忆的, 另外还有一个称为时间尺度不变性(time-scale invariance)的有用特性。例 3.5 中, 输入信号作为一个时间函数可以表示为:

$$temperature : \mathbb{R} \rightarrow \{absent\} \cup \mathbb{R}$$

因此  $temperature(t)$  为  $t$  时刻的温度读数, 如果该时刻没有温度读数, 则为  $absent$ 。其输出也是时间函数, 表示为:

$$heatOn, heatOff : \mathbb{R} \rightarrow \{absent, present\}$$

假设替代  $temperature$ ,  $\alpha > 0$  时输入表示为:

$$temperature'(t) = temperature(\alpha \cdot t)$$

当  $\alpha > 1$ , 则输入随时间变化比较快, 而当  $\alpha < 1$ , 则输入变化比较慢。不过, 这两种情形下, 输入模式都是一样的。因此, 对于 FSM, 输出  $heatOn'$  和  $heatOff'$  可表示为:

$$heatOn'(t) = heatOn(\alpha \cdot t) \quad heatOff'(t) = heatOff(\alpha \cdot t)$$

时间尺度不变性是指输入时间轴上的缩放比例决定输出时间轴的比例, 所以绝对的时间尺度是不相关的。

恒温器的另一种实现使用单一的温度阈值, 这样无论温度如何, 都不需要加热器至少保持开或关一段时间。这样的设计效果参见练习 2。

当外部条件决定状态机进行响应时, 输入有一个估值。状态机分配一个估值到输出端口, 并且(可能)改变到一个新的状态。如果从当前状态的任何转移中没有判定式的值为真, 则状态机将保持相同状态。

响应时可能所有的输入都为 *absent*。即使在这种情形下, 判定式的估值也可能为真, 这时会有转移发生。如果输入为 *absent*, 并且从当前状态的任何转移中没有判定式估值为真, 则状态机将发生阻塞(stutter)。阻塞响应是输入和输出都为 *absent* 并且状态机不改变状态时的响应。没有进展, 而且没有变化。

**例 3.6:** 在图 3-4 中, 如果响应时两个输入都不存在, 则状态机将阻塞。如果处于状态 0, 且输入 *down* 为 *present*, 则只有向外转移的判定式为假, 状态机保持相同状态。然而, 这时不是一个阻塞响应, 因为输入并非都为 *absent*。

例 3.1 所示停车场计数器的非正式描述没有明确说明当计数为 0 且有一辆车离开时会发生什么。FSM 模型的主要优点是可以定义所有可能的行为。图 3-4 所示的模型定义这种情形下将发生什么。计数保持为 0。因此, FSM 模型进行正式检查, 确定一些特定行为是否是实际上期望的行为。非正式的说明不适用于这样的测试, 至少不能完全适用。

虽然看起来图 3-4 中的模型没有定义状态为 0 且 *down* 为 *present* 时会发生什么, 但它隐含地给出了定义——状态保持不变且不产生输出。该响应在图中没有明确地显示。有时需要强调这些响应, 这种情形下可以明确显示出来。简单的做法采用**默认转移**, 如图 3-6 所示。图中, 默认转移用虚线表示, 且用“*true*/”标注。当没有非默认转移发生且判定式的值为 *true* 时, 有默认转移发生。因此图 3-6 中, 如果  $up \wedge \neg down$  的结果为 *false*, 则有默认转移发生, 且默认转移时输出为 *absent*。

默认转移可以给出简单的标注, 但并非必需的。任何默认转移都可以被一个具有恰当判定式的普通转移所替代。例如, 在图 3-6 中, 可以采用带有判定式  $\neg(up \wedge \neg down)$  的普通转移。

普通转移和默认转移在图中的使用可以看做一种转移的优先分配方式。普通转移有着高于默认转移的优先级。当两个判定式都为 *true* 时, 以普通转移为准。有些状态机形式支持两层以上的优先级。例如, SyncCharts(André, 1996)让每个转移对应一个整数优先级。这使判定式表达更简单, 不过需要在图示中标明优

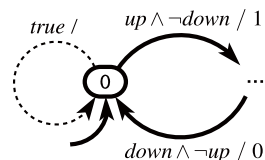


图 3-6 默认转移(由于它返回同一状态且不产生输出, 所以不需要明确显示)

优先级。

### 3.3.3 升级函数

FSM 的图形表示定义的是一种状态机动态数学模型。一个与图形表示具有相同含义的数学符号有时可以更为简洁，尤其是对于大型状态机，图形表示变得非常累赘。在数学符号中，有限状态机为一个五元组：

$$(States, Inputs, Outputs, update, initialState)$$

其中

- $States$ (状态)是一个状态的有限集；
- $Inputs$ (输入)是一个输入值集；
- $Outputs$ (输出)是一个输出值集；
- $update$ (升级)： $States \times Inputs \rightarrow States \times Outputs$  是一个升级函数(*update function*)，将一个状态和一个输入值映射到下一状态和输出值；
- $initialState$ (初始状态)是初始状态。

FSM 在一系列响应中进行响应。在每个响应里，FSM 有一个当前状态，且该响应可能转移到下一状态，成为下一个响应的当前状态。设初始状态为 0，可以将这些状态进行编号。具体来说，设  $s: \mathbb{N} \rightarrow States$  为一个函数，给出响应  $n \in \mathbb{N}$  时的 FSM 状态。最初， $s(0) = initialState$ 。

设  $x: \mathbb{N} \rightarrow Inputs$  且  $y: \mathbb{N} \rightarrow Outputs$  表示每个响应的输入值和输出值。这样一来， $x(0) \in Inputs$  是第一个输入值而  $y(0) \in Outputs$  是第一个输出值。状态机的动态由下式给出：

$$(s(n+1), y(n)) = update(s(n), x(n)) \quad (3.1)$$

#### 支持 FSM 的软件工具

FSM 长时间被用于计算机科学理论和软件工程(Hopcroft and Ullman, 1979)。已有大量的软件工具支持有限状态机的设计和分析。状态图(Statechart) (Harel, 1987)，一种针对分层 FSM 的并发结构的表示法，给很多这样的工具带来了影响。STATEMATE (Harel et al., 1990)是第一个支持状态图表示法的工具，随后演变为 Rational Rose，由 IBM 销售。现在有很多状态图的改进版(von der Beeck, 1994)，其中有些得到几乎所有提供统一建模语言(Unified Modeling Language, UML)功能(Booch et al., 1998)的软件工程工具支持。SyncCharts(André, 1996)是一个特别出色的改进版，它借鉴了 Esterel (Berry and Gonthier, 1992)用于 FSM 并发结构的严格语义。LabVIEW 支持可以操作数据流图的状态图改进版，而带有状态流扩展的 Simulink 支持可以操作连续时间模型的改进版。

根据当前状态和输入给出下一状态和输出。 $update$  函数对 FSM 中所有的转移、判定式和输出规格进行编码。术语转移函数通常用来代替升级函数。

输入值和输出值也有一种固有的数学形式。假设 FSM 具有输入端口  $P = \{p_1, \dots, p_N\}$ ，其中每个  $p \in P$  有一个相应的类型  $V_p$ 。那么  $Inputs$  是形如下式的函数集：

$$i: P \rightarrow V_{p_1} \cup \dots \cup V_{p_N} \cup \{absent\}$$

其中，对于每个  $p \in P$ ， $i(p) \in V_p \cup \{absent\}$  给出端口  $p$  的值。因此，函数  $i \in Inputs$  是输入

端口的值。

例 3.7：图 3-4 中的 FSM 可以利用数学方式表示为：

$$\begin{aligned} States &= \{0, 1, \dots, M\} \\ Inputs &= (\{up, down\} \rightarrow \{present, absent\}) \\ Outputs &= (\{count\} \rightarrow \{0, 1, \dots, M, absent\}) \\ initialState &= 0 \end{aligned}$$

对于所有  $s \in States$  和  $i \in Inputs$ ，升级函数为：

$$update(s, i) = \begin{cases} (s+1, s+1) & \text{如果 } s < M \\ & \wedge i(up) = present \\ & \wedge i(down) = absent \\ (s-1, s-1) & \text{如果 } s > 0 \\ & \wedge i(up) = absent \\ & \wedge i(down) = present \\ (s, absent) & \text{否则} \end{cases} \quad (3.2)$$

值得注意的是，输出值  $o \in Outputs$  是一个形式为  $o : \{count\} \rightarrow \{0, 1, \dots, M, absent\}$  的函数。在式(3.2)中，第一个选择给出输出值  $o = s+1$ ，称为常量函数，即对于所有的  $q \in Q = \{count\}$  都有  $o(q) = s+1$ 。当输出端口多于一个时，需要明确哪个输出值分配给哪个端口。这种情形下，可以使用与图中表示动作相同的符号。

### 3.3.4 确定性和可接受性

本节中介绍的状态机具有两个重要特性：

**确定性**(Determinacy)：如果对于每个状态，每个输入值最多可激活一个转移，则称这样的状态机具有确定性。由于  $update$  是一个函数，而非一对多的映射，所以上面给出的 FSM 的形式定义确保它是确定的。然而，对在转移上有判定式的图形符号没有这样的约束。只有当离开每个状态的判定式都是不重叠的，这样的状态机才会被认为是确定的。

**可接受性**(Receptiveness)：如果对于每个状态，每个输入符号都有至少一个可能的转移，则称这样的状态机为可接受的。换句话说，可接受性确保状态机对任何输入准备随时响应，且在状态都不出现“阻塞”。由于  $update$  是一个函数，而非一个部分函数(partial function)，所以上面给出的 FSM 的形式定义确保它是可接受的。它定义了每个可能的状态和输入值。此外，由于有一些隐含的默认转移，所以在图形符号中指定的所有状态机都是可接受的。

由此可见，如果状态机既是确定的又是可接受的，则对于每个状态，都有一个可能的转移对应一个输入值。

## 3.4 扩展状态机

当状态的数目变大时，FSM 的标注将变得非常不方便。图 3-4 所示的停车场计数器清晰地说明了这点。如果  $M$  很大，这种圈和弧的表示将变得很不实用，这就是为什么图中采用了“...”的省略表达形式。

**扩展状态机**(extended state machine)可以利用变量扩大 FSM 模型来解决这个问题，将变量作为状态间转移的一部分来读写。

例 3.8: 图 3-4 所示的停车场计数器, 可以利用如图 3-8 所示的扩展状态机简单表示。

图中显示了一个变量  $c$ , 在左上角明确地声明  $c$  是一个变量而非输入或输出。表示初始状态的转移将这个变量初始化为 0。

上端的自环转移发生在输入  $up$  为 *present*、 $down$  为 *absent* 且变量  $c$  小于  $M$  时。当转移发生时, 状态机产生一个值为  $c+1$  的输出  $count$ , 然后  $c$  的值递增 1。

下端的自环转移发生在输入  $down$  为 *present*、 $up$  为 *absent* 且变量  $c$  大于 0 时。一旦转移发生, 状态机产生一个值为  $c-1$  的输出,  $c$  的值递减 1。

### Moore 机和 Mealy 机

本章中讲述的状态机通常称为 Mealy 机, 是以贝尔实验室的一位工程师 George H. Mealy 的名字命名的, 他在 1955 年发表了这些状态机的描述 (Mealy, 1955)。Mealy 机的特点是有转移时产生输出。另一类状态机——Moore 机是在状态中产生输出, 而不是在发生转移时。也就是说, 输出是由当前状态而不是当前转移所定义。Moore 机是以贝尔实验室的另一位工程师 Edward F. Moore 的名字命名的, 他在 1956 年发表的论文 (Moore, 1956) 中描述了它们。

这些状态机之间的区别是微妙而重要的。两者都是离散系统, 因此它们的操作是由一系列离散响应组成。对于 Moore 机来说, 响应时产生的输出由当前状态所定义 (在响应开始时, 而不在结束时)。因此, 响应时的输出不依赖于同一时间的输入。输入决定发生哪个转移, 但不决定响应产生什么输出。因此, Moore 机是严格因果关系的。

停车场计数器的 Moore 机版如图 3-7 所示。利用一个斜线符号将输出显示在状态中, 而不是在转移上。然而值得注意的是, 该状态机并不等同于图 3-4 所示的状态机。例如, 假设第一个响应时,  $up = present$  且  $down = absent$ 。此时的输出在图 3-7 中为 0, 而在图 3-4 中为 1。Moore 机的输出表示有一辆车到达时停车场中车的数量, 而不是车到达后车的数量。假设第一个响应时,  $up = down = absent$ 。此时图 3-7 中的输出为 0, 图 3-4 的输出为 *absent*。响应时, Moore 机始终报告与当前状态相关的输出。Mealy 机并不产生输出, 除非有转移明确给出那个输出。

任何 Moore 机都可以转换为一个等效的 Mealy 机。Mealy 机可以转换成大致等效的 Moore 机, 差别只是输出产生于下一个响应, 而非当前响应。由于 Mealy 机往往更加精简 (需要较少的状态来表示同样的功能), 而且能够即时对输入产生输出, 所以我们采用 Mealy 机。

inputs:  $up, down$ : pure  
output:  $count$ :  $\{0, \dots, M\}$

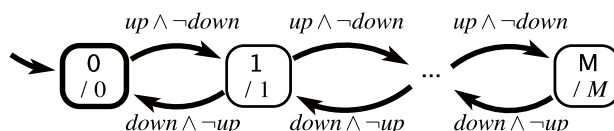
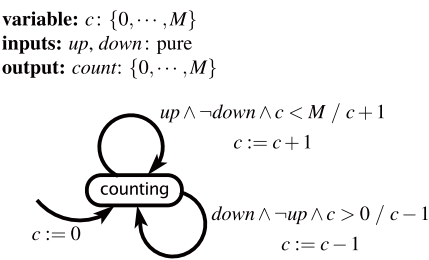


图 3-7 停车场中车数量记录系统的 Moore 机 (注意该状态机不等同于图 3-4 所示的状态机)

值得注意的是,  $M$  是一个参数, 而不是一个变量。具体来说, 它在执行过程中被假定为常量。



个离散状态(圈)和  $m$  个变量, 每个变量取  $p$  个可能值中的一个, 则状态机的状态空间大小为:

$$|States| = np^m$$

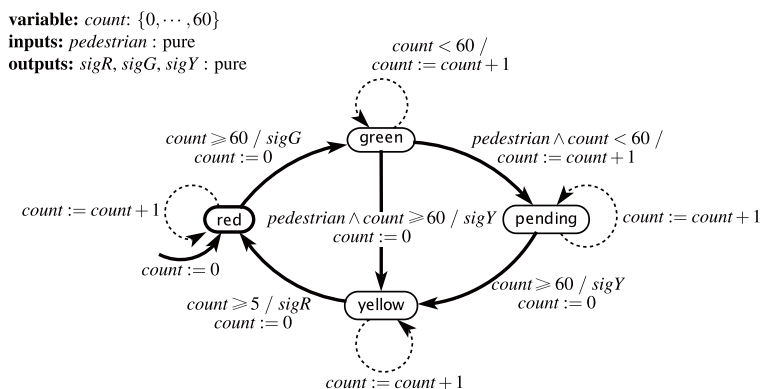


图 3-10 交通灯控制器的扩展状态机模型(跟踪时间推移, 假定以固定的时间间隔进行响应)

例 3.10: 图 3-8 中的停车场计数器为  $n = 1$ ,  $m = 1$ , 且  $p = M + 1$ , 所以总的状态数为  $M + 1$ 。

扩展状态机可以是 FSM, 也可以不是。特别地,  $p$  为无穷大的情形也并非罕见。例如, 一个变量的值可以是自然数  $\mathbb{N}$  中的任何一个, 那么这种情形下的状态就是无穷的。

例 3.11: 如果将图 3-8 中的状态机修改一下, 上端转移的判定式改为:

$$up \wedge \neg down$$

而不是

$$up \wedge \neg down \wedge c < M$$

那么这个状态机不再是一个有限状态机。

有些状态机包括一些无法到达的状态, 因此可到达状态(reachable state)集——由所有从初始状态通过输入序列可以到达的状态组成——可能比状态集小。

例 3.12: 尽管图 3-10 只有四个圈, 但事实上状态数目却大得多。变量  $count$  有 61 个可能的值且有 4 个圈, 所以组合后的总数为  $61 \times 4 = 244$ 。因此状态空间的大小为 244。然而, 并不是所有这些状态都是可到达的。特别是, 在黄灯的状态下, 变量  $count$  仅有 6 个值  $\{0, \dots, 5\}$  中的一个。因此, 可到达状态数为  $61 \times 3 + 6 = 189$ 。

### 3.5 非确定性

大多数令人感兴趣的状态机都是响应输入, 产生输出。这些输入应该来自某处, 其输出应该去往某处。这里“某处”称为状态机的外部环境(environment)。

例 3.13: 图 3-10 所示的交通灯控制器有一个纯输入信号  $pedestrian$ 。行人到达人行横道时, 输入为  $present$ 。交通灯保持为绿灯, 直到有行人到达。有些子系统负责生成  $pedestrian$  事件, 由行人按下一个按钮请求通行。另一些子系统则成为图 3-10 所示 FSM 外部环境的一部分。

问题是如何构建外部环境模型。以交通灯为例，可以为此构建一个城市人流模型。不过，这将是一个非常复杂的模型，而且可能比实际需要更加详细。想要忽略无关紧要的细节，只关注交通灯的设计，那么可以利用非确定性状态机来实现。

**例 3.14:** 图 3-11 所示的 FSM 对有行人到达装有如图 3-10 所示的交通灯控制器的人行横道进行建模。FSM 有三个输入，它们应该都来自于图 3-10 的输出。其唯一输出 *pedestrian* 成为图 3-10 的(下一个)输入。

初始状态为 *crossing* (为什么? 参见练习 4)。当 *sigG* 被接受，FSM 转移到 *none*。从该状态引出的两个转移的判定式都为 *true*，表示这两个转移都有发生。由于有两个转移发生，所以这个状态机是非确定的。FSM 可以保持同一状态，不产生输出，或转移到 *waiting* 并产生输出 *pedestrian*。

该状态机与图 3-10 所示状态机之间的互动极其微妙。这种设计上的变化将在练习 4 考虑，第 6 章将详细介绍两种状态机的组合。

如果对于状态机的任何一个状态，都有两个不同的转移，同一个响应的判定式的值都为 *true*，那么称这个状态机是**非确定的** (nondeterminate)，或是**具有非确定性的** (nondeterministic)。在表示这种状态机的图中，使状态机非确定的转移可以用红色。如图 3-11 所示，退出状态 *none* 的转移是使此状态机非确定的部分。

还可以定义具有多个初始状态的状态机。这样的状态机也是非确定的。练习 4 中将给出一个实例以供思考。

在这两种情形下，一个非确定的 FSM 指定一组可能的响应，而不是单一的响应。操作上，所有响应都是可能的。这种非确定性 FSM 不判断各个响应有多大可能。比如在图 3-11 中，一直进行状态 *none* 的自环转移是完全正确的。指定似然性的模型(以概率形式)是一种**随机模型** (stochastic model)，与非确定性模型有着相当大的差异。

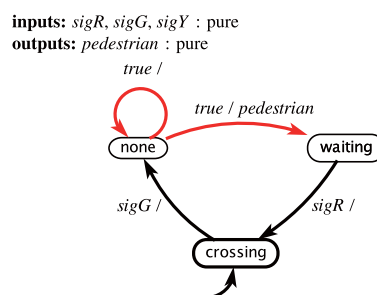


图 3-11 到达人行横道的非确定性模型

### 3.5.1 形式化模型

形式上，**非确定性有限状态机** (nondeterministic FSM) 与确定性 FSM 一样，可以表示成一个五元组：

$$(States, Inputs, Outputs, possibleUpdates, initialStates)$$

前三个元素与确定性 FSM 相同，但后两个是不同的：

- *States* 为一个有限状态集；
- *Inputs* 为输入值的集合；
- *Outputs* 为输出值的集合；
- $possibleUpdates : States \times Inputs \rightarrow 2^{States \times Outputs}$  为**升级关系** (update relation)，将状态和输入值映射到可能的(下一状态，输出值)对集；
- *initialStates* 为初始状态的集合。

函数 *possibleUpdates* 的形式表明对于给定的当前状态和输入值，可以有多个下一状态和/或输出值。值域为  $States \times Outputs$  次幂集。将 *possibleUpdates* 函数作为一个升级关系来考虑，以强调这种差异。术语**转移关系** (transition relation) 也常用来代替升级关系。

事实上，非确定性 FSM 可以有多个初始状态，因此  $initialStates$  是一个集合，而不是状态集  $States$  中的一个元素。

例 3.15：图 3-11 中的有限状态机形式上可以表示为：

$$States = \{none, waiting, crossing\}$$

$$Inputs = (\{sigG, sigY, sigR\} \rightarrow \{present, absent\})$$

$$Outputs = (\{pedestrian\} \rightarrow \{present, absent\})$$

$$initialStates = \{crossing\}$$

对于所有  $s \in States$  和  $i \in Inputs$ ，升级关系为：

$$possibleUpdates(s, i) = \begin{cases} \{(none, absent)\} & \text{如果 } s=crossing \\ & \wedge i(sigG) = present \\ \{(none, absent), (waiting, present)\} & \text{如果 } s=none \\ \{(crossing, absent)\} & \text{如果 } s=waiting \\ & \wedge i(sigR) = present \\ \{(s, absent)\} & \text{否则} \end{cases} \quad (3.3)$$

值得注意的是，输出值  $o \in Outputs$  是一个形式如  $o : \{pedestrian\} \rightarrow \{present, absent\}$  的函数。在式(3.3)中，第二种选择给出两种可能的结果，反映了状态机的非确定性。

### 3.5.2 非确定性的用途

非确定性自身是一个有趣的数学概念，在嵌入式系统建模中有两个主要的用途：

**环境建模：**它对于隐藏与环境如何运作无关的细节非常有用，从而形成一个非确定性 FSM 模型。图 3-11 所示的就是这样一个环境建模实例。

**规格：**系统规格对系统特性强加一些限制，而对另一些特性没作限制。非确定性对于这样的设置也是一个有用的建模方法。例如，有这样一个交通灯规格，以红、绿、黄的顺序循环，且不考虑各个输出之间的时间。图 3-12 所示的非确定性 FSM 对此规格进行建模。每个转移上的判定式值为  $true$  表明转移可以在任何步骤中发生。从技术上来说，这意味着对于  $inputs$  中的任何输入值都有转移发生。

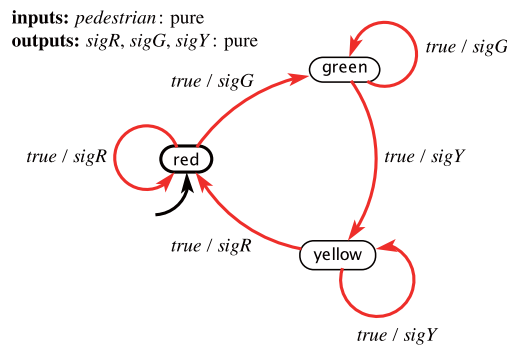


图 3-12 非确定性 FSM(指定信号灯顺序而不是它们变亮的时间。这里忽略  $pedestrian$  输入)

### 3.6 行为和轨迹

FSM 具有离散动态。如 3.3.3 节所述，可以将时间的推移抽象出来，只考虑响应序列，忽略每个响应发生的时间。不需要讨论响应之间所需的时间量，因为事实上这与 FSM 的行为毫不相关。

考虑一个  $V_p$  型状态机的一个端口  $p$ 。该端口拥有存在于集合  $V_p \cup \{absent\}$  中的一系列值，每个响应对应一个值。可以用以下形式的函数来表示这个序列：

$$s_p : \mathbb{N} \rightarrow V_p \cup \{absent\}$$

这是端口接收到的信号(如果是一个输入)或在该端口上产生的信号(如果是一个输出)。

状态机的行为是将信号分配到每个端口，使每个输出端口的信号是对于给定的输入信号所产生的输出序列。

**例 3.16：**图 3-4 中的停车场计数器有输入端口集合  $P = \{up, down\}$ ，其类型为  $V_{up} = V_{down} = \{present\}$ ，以及输出端口集合  $Q = \{count\}$ ，其类型为  $V_{count} = \{0, \dots, M\}$ 。例如，输入序列为：

$$\begin{aligned} s_{up} &= (present, absent, present, absent, present, \dots) \\ s_{down} &= (present, absent, absent, present, absent, \dots) \end{aligned}$$

相应的输出序列为：

$$s_{count} = (absent, absent, 1, 0, 1, \dots)$$

这三个信号  $s_{up}$ 、 $s_{down}$  以及  $s_{count}$  合在一起构成状态机的一个行为。如果设

$$s'_{count} = (1, 2, 3, 4, 5, \dots)$$

那么， $s_{up}$ 、 $s_{down}$  以及  $s'_{count}$  合在一起不是状态机的一个行为。信号  $s'_{count}$  并不是由那些输入的响应所产生的。

确定性状态机具备这样一个特性，对于每个输入序列集，严格地只有一个行为与之对应。也就是说，如果已知输入序列，则输出序列完全确定。这样的状态机可以看做一个将输入序列映射到输出序列的函数。非确定状态机可以有多个行为共享同一个输入序列，因而不能将其看做一个输入序列映射到输出序列的函数。

状态机  $M$  的所有行为的集合称为它的**语言**，写做  $L(M)$ 。由于状态机是可接受的，因此它们的语言通常包括所有有可能的输入序列。

行为可以更方便地表示为一系列值，称为**可观察轨迹**(observable trace)。在响应  $i$  中，设  $x_i$  表示输入端口的值， $y_i$  表示输出端口的值。可观察轨迹为序列：

$$((x_0, y_0), (x_1, y_1), (x_2, y_2), \dots)$$

可观察轨迹实际上只是行为的另一种表达。

推理在行为中遍历的各个状态往往很有用。**执行轨迹**(execution trace)包括状态轨迹，可以写成序列：

$$((x_0, s_0, y_0), (x_1, s_1, y_1), (x_2, s_2, y_2), \dots)$$

其中  $s_0 = initialState$ 。可以更形象地表示如下：

$$s_0 \xrightarrow{x_0/y_0} s_1 \xrightarrow{x_1/y_1} s_2 \xrightarrow{x_2/y_2} \dots$$

如果对于所有  $i \in \mathbb{N}$ ，都有  $(s_i + 1, y_i) = update(s_i, x_i)$  (对于确定性状态机而言)，或者都

有  $(s_i+1, y_i) \in possibleUpdates(s_i, x_i)$  (对于非确定性状态机而言), 那么这是一个执行轨迹。

**例 3.17:** 再次考虑图 3-4 中的停车场计数器, 输入序列  $s_{up}$  及  $s_{down}$  与例 3.16 相同。相应的执行轨迹可以写为:

$$0 \xrightarrow{up \wedge down /} 0 \xrightarrow{/} 0 \xrightarrow{up / 1} 1 \xrightarrow{down / 0} 0 \xrightarrow{up / 1} \dots$$

这里, 对用于 3.3.1 节转移上的值采用相同的简写。例如, 标记“ $up / 1$ ”表示  $up$  的值为 *present*,  $down$  的值为 *absent*, 且  $count$  值为 1。每个明确而无歧义地表示输入值和输出值的符号都是可接受的。

对于非确定性状态机, 表示特定的输入序列所对应的所有可能轨迹, 甚至表示从所有可能的输入序列所产生的所有可能的轨迹都是有效的。可以使用 **计算树** (computation tree) 实现。

**例 3.18:** 考虑图 3-12 中的非确定性 FSM。图 3-13 所示的是任意输入序列的前三个响应所对应的计算树。树中的节点表示状态, 边用输入和输出的值来标记。其中符号 *true* 表示任意输入值。

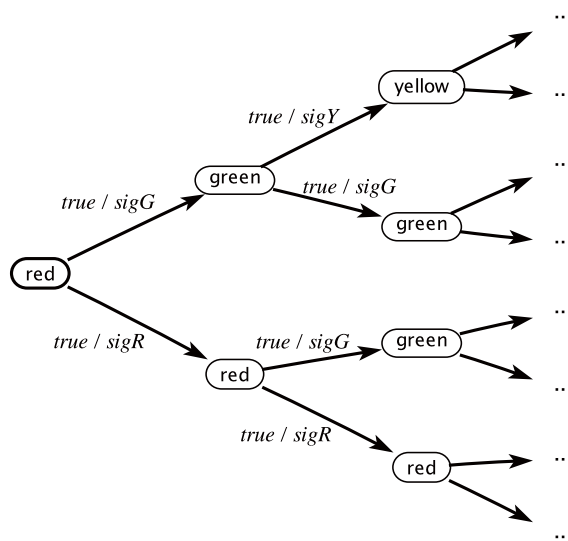


图 3-13 图 3-12 所示 FSM 的计算树

轨迹 (trace) 以及计算树可以用来进一步理解状态机的行为, 并进行验证以避免不合要求的行为。

### 3.7 小结

本章介绍了使用状态机对离散动态系统建模。给出了适用于有限状态机的图形符号, 以及可以简洁地表示大量状态的扩展状态机符号。还介绍了利用集合与函数来代替视觉符号的数学模型。数学符号可以有效地确保一个模型的精确解释, 还能证明模型的一些特性。本章还讨论了非确定性, 可以对简单表示的多组行为进行合理抽象。

## 练习

1. 考虑一个事件计数器，作为 3.1 节计数器的简化版本。图标如下：



这个参量开始于状态  $i$ ，一旦有一个事件到达输入端口，那么递增状态并将更新的值发送到输出端口。因此， $e$  是一个纯信号， $c$  的形式为  $c: \mathbb{R} \rightarrow \{\text{absent}\} \cup \mathbb{N}$ ，假设  $i \in \mathbb{N}$ 。假设将这个事件计数器用于气象站计算温度超过某个阈值的次数。请生成事件计数器的一个合理的输入信号  $e$ 。可以创建多个版本。对于所有版本，设计一个状态机，其输入为信号  $\tau: \mathbb{R} \rightarrow \{\text{absent}\} \cup \mathbb{Z}$ ，每小时给出一次当前温度(摄氏度)。输出为一个到达事件计数器的纯信号  $e: \mathbb{R} \rightarrow \{\text{absent}, \text{present}\}$ 。

- (a) 对于第一个版本，每当输入为 *present* 且温度高于 38 度时，状态机只产生一个 *present* 输出；否则，输出为 *absent*。
- (b) 对于第二个版本，状态机应该具有滞后功能。具体来说，第一次输入大于 38 度时应该产生一个 *present* 输出。随后，每当输入高于 38 度都应该产生一个 *present* 输出，但是自从产生最后一个 *present* 输出，输入已经降低到 36 度以下。
- (c) 对于第三个版本，状态机应该实现与 (b) 部分相同的滞后，但每天最多产生一次 *present* 输出。
2. 考虑例 3.5 中恒温器的改进版。在这个改进版中，只有一个温度阈值。为了避免抖动，恒温器使加热器在至少一个固定的时间内保持开或关状态。在初始状态，如果温度小于或等于 20 摄氏度，开启加热器，并至少持续 30 秒。然后，如果温度高于 20 度，将加热器关闭，并持续至少 2 分钟。仅当温度低于或等于 20 度时，再开启加热器。
- (a) 设计一个 FSM，使其与上述行为相同，假设每 30 秒只响应一次。
- (b) 恒温器会有多少种可能状态？这是最少的可能状态数吗？
- (c) 该恒温器模型具有时间尺度不变性吗？
3. 考虑图 3-14 所示的确定性有限状态机，它是交通灯的简单模型。
- (a) 以五元组的形式描述这个 FSM：  
 $(\text{States}, \text{Inputs}, \text{Outputs}, \text{update}, \text{initialState})$
- (b) 给出 FSM 的一个长度为 4 的执行轨迹，假设输入节拍为在每个响应上出现。
- (c) 考虑合并 *red* 和 *yellow* 状态为单一的 *stop* 状态。进入或离开原 *red* 和 *yellow* 状态的转移都直接进入或离开 *stop* 状态。其他的转移以及输入和输出都保持不变。*stop* 状态为新的初始状态。这样生成的状态机是确定性的吗？为什么？如果是确定性的，给出一个长度为 4 的轨迹前缀。如果是非确定性的，画出深度为 4 的计算树。

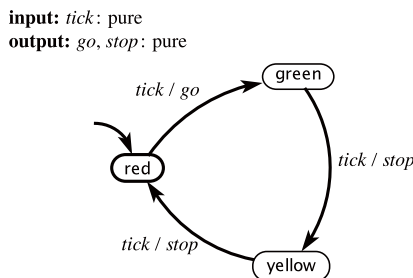
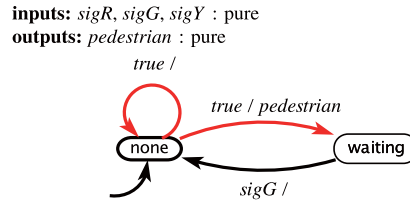


图 3-14 练习 3 的确定性有限状态机

4. 这个问题是要考虑图 3-11 所示的 FSM 的改进版，对到达人行横道的行人进行建模。假设人行横道的交通灯由图 3-10 所示的 FSM 控制。在所有的情形下，假设是一个时间触发模型，行人模型和交通灯模型

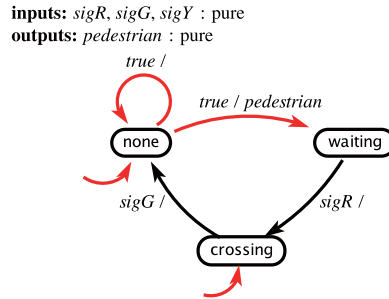
每秒响应一次。进一步假设在每一个响应，每个状态机都将输入看做是其他状态机在同一响应中产生的输出(这种形式的组合称为同步组合，将在第6章进一步研究)。

(a) 假设用下面的 FSM 对行人到达进行建模，用以替代图 3-11:



给出行人到达(上面的状态机转移到 **waiting** 状态)但不让其通行的轨迹。也就是说，行人到达之后，交通灯处于 **red** 状态。

(b) 假设用下面的 FSM 对行人到达进行建模，用以替代图 3-11:



这里，初始状态非确定地选择为 **none** 或者 **crossing**。给出行人到达(上面的状态机从 **none** 转移到 **waiting** 状态)但不让他通行的轨迹。也就是说，行人到达之后，交通灯处于 **red** 状态。

5. 考虑图 3-15 所示的状态机。判断下面的每行是否是状态机的行为。下面每行中的省略号“...”是指最后的符号一直重复。当然，为了阅读方便，*absent* 缩写为 *a*，*present* 缩写为 *p*。

(a)  $x = (p, p, p, p, p, \dots)$ ,  $y = (0, 1, 1, 0, 0, \dots)$

(b)  $x = (p, p, p, p, p, \dots)$ ,  $y = (0, 1, 1, 0, a, \dots)$

(c)  $x = (a, p, a, p, a, \dots)$ ,  $y = (a, 1, a, 0, a, \dots)$

(d)  $x = (p, p, p, p, p, \dots)$ ,  $y = (0, 0, a, a, a, \dots)$

(e)  $x = (p, p, p, p, p, \dots)$ ,  $y = (0, a, 0, a, a, \dots)$

input:  $x$ : pure  
output:  $y$ :  $\{0, 1\}$

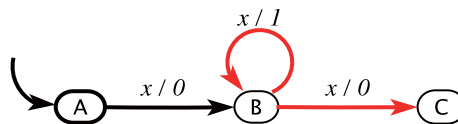


图 3-15 练习 5 的状态机

6. (注意：这道练习题有些难度)研究本章前面补充资料“进一步探讨：离散信号”中定义的离散信号的特性。具体来说，是要说明离散性并不是一个组成(compositional)特性。也就是说，把单一系统的两个离散行为组合在一起，组合结果不一定是离散的。

(a) 考虑一个纯信号  $x: \mathbb{R} \rightarrow \{present, absent\}$ ，表示为：

$$x(t) = \begin{cases} present & \text{如果 } t \text{ 是一个非负整数} \\ absent & \text{否则} \end{cases}$$

其中,  $t \in \mathbb{R}$ 。请说明这个信号是离散的。

(b) 考虑一个纯信号  $y: \mathbb{R} \rightarrow \{\text{present}, \text{absent}\}$ , 表示为:

$$y(t) = \begin{cases} \text{present} & \text{如果对于任意的正整数 } n, t = 1 - 1/n \\ \text{absent} & \text{否则} \end{cases}$$

其中,  $t \in \mathbb{R}$ 。请说明这个信号是离散的。

(c) 考虑信号  $w$  为前两部分中  $x$  和  $y$  的组合。也就是说, 当  $x(t) = \text{present}$  或者  $y(t) = \text{present}$  时,  $w(t) = \text{present}$ ; 否则为  $\text{absent}$ 。请说明  $w$  不是离散的。

(d) 考虑图 3-1 所示的实例。假设 *arrival* 和 *departure* 这两个信号都是离散的。请说明这并不意味着输出 *count* 是一个离散信号。

